Crat3r
#03 - Images, Synchronization and the Swap-Chain

Erik Eismann, Marc Stamminger

October 27, 2026

Last Time

- Create compute pipelines to dispatch compute shaders
- Push constants to bring small uniform data to shader, descriptors to make resources (buffers / images) available
- Buffers: create buffer, allocate memory of correct type, then bind together

Images

Images

- Images are multidimensional arrays of texels.
- Usage: textures, storage images, render targets, depth buffers, …
- Compared to buffers, images have additional properties:
  - In addition to their three base dimensions: **mip levels** and **array layers**.
  - A **format** that describes the data of a single texel (<1 to multiple bytes)
  - A **tiling** that describes the arrangement in memory
  - A **layout** that describes the current internal state of the image
- Shaders do not access images directly, but instead use **image views** and **samplers**.

Image Creation

```
VkResult vkCreateImage(
    VkDevice                        device,
    const VkImageCreateInfo*        pCreateInfo,
    const VkAllocationCallbacks*    pAllocator,
    VkImage*                        pImage);
```

```
typedef struct VkImageCreateInfo {
    VkStructureType                 sType;
    const void*                     pNext;
    VkImageCreateFlags              flags;
    VkImageType                     imageType;
    VkFormat                        format;
    VkExtent3D                      extent;
    uint32_t                        mipLevels;
    uint32_t                        arrayLayers;
    VkSampleCountFlagBits           samples;
    VkImageTiling                   tiling;
    VkImageUsageFlags               usage;
    VkSharingMode                   sharingMode;
    uint32_t                        queueFamilyIndexCount;
    const uint32_t*                 pQueueFamilyIndices;
    VkImageLayout                   initialLayout;
} VkImageCreateInfo;
```

- `sharingMode` and queue family indices work exactly as for buffers.
- `initialLayout` must be either `VK_IMAGE_LAYOUT_UNDEFINED` or `VK_IMAGE_LAYOUT_PREINITIALIZED`.
- `samples` is `VK_SAMPLE_COUNT_1_BIT` unless multisampling is used.

Image Type and Extent

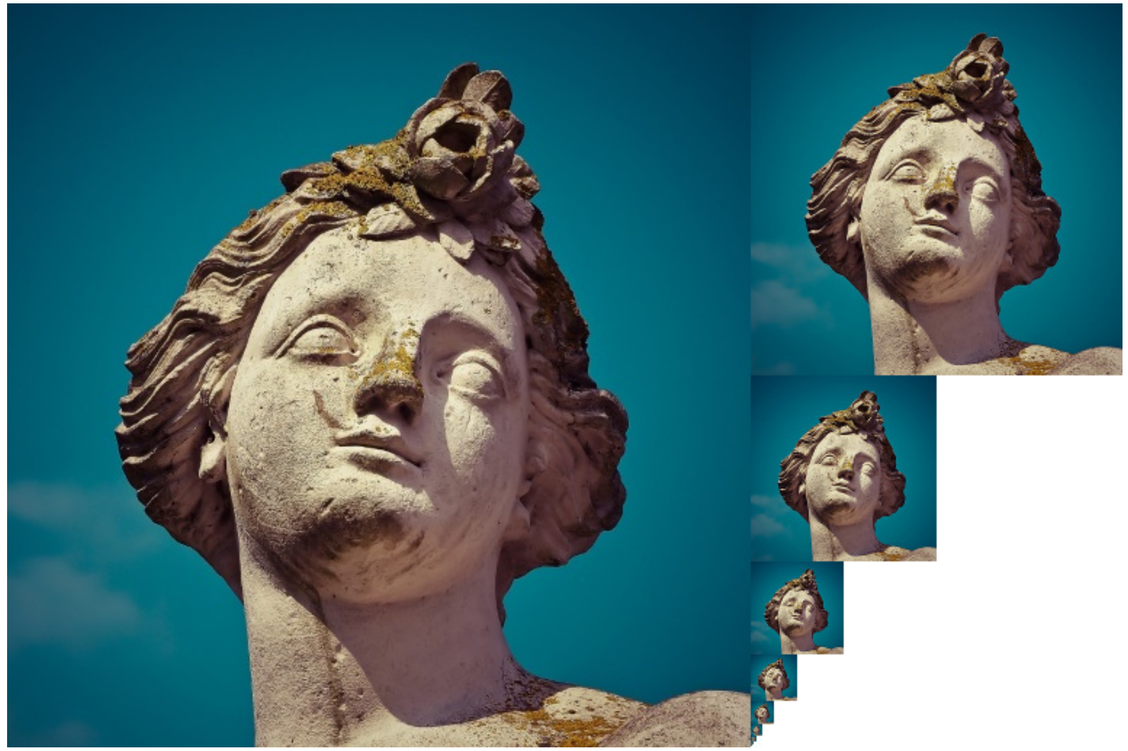
```
    VkImageType                 imageType;
    VkExtent3D                  extent;
```

```
typedef enum VkImageType {
    VK_IMAGE_TYPE_1D = 0,
    VK_IMAGE_TYPE_2D = 1,
    VK_IMAGE_TYPE_3D = 2,
} VkImageType;

typedef struct VkExtent3D {
    uint32_t    width;
    uint32_t    height;
    uint32_t    depth;
} VkExtent3D;
```

- If `imageType` is `VK_IMAGE_TYPE_1D`, both `height` and `depth` **must** be 1.
- If `imageType` is `VK_IMAGE_TYPE_2D`, `depth` **must** be 1.
- Maximum dimensions:
  Vulkan requires 4096 for 1D / 2D images and 256 for 3D images. Most GPUs support 16384 and 2048 respectively.

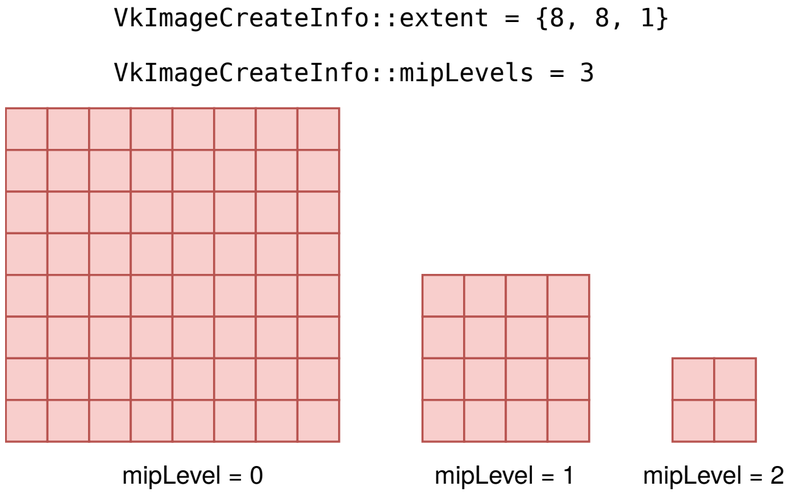
Mipmaps

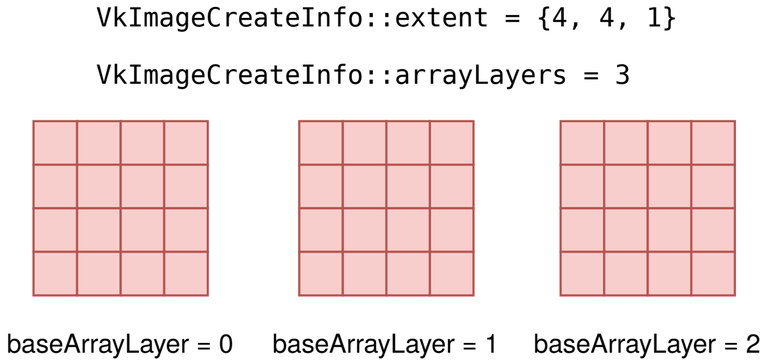


https://vulkan-tutorial.com/Generating_Mipmaps

- Multiple levels of downsampled images
- Reduces aliasing and improves cache locality on distant objects
- Each mip level is half the size of the previous level in every dimension
- `floor(log2(max(w, h, d))) + 1` levels total
- Set in `mipLevels` create info field
- Last level is 1 texel in every dimension
- Adds approximately 33% storage size for 2D
- Mipmaps are not generated automatically

Array Layers



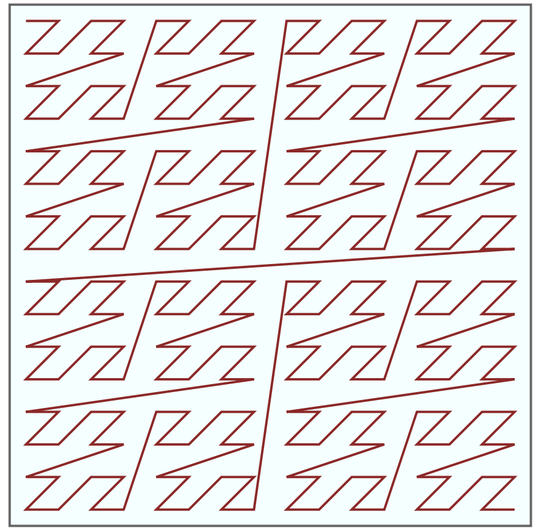


https://vulkan.lunarg.com/doc/view/1.4.328.1/windows/antora/guide/latest/image_copies.html

- Multiple images of same dimensions and format packed into one `VkImage`
- Use cases:
  - batched material textures
  - animated textures
  - shadow map cascades
  - cubemaps
- Cubemap requirements:
  - flag `VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT`
  - `imageType` must be `VK_IMAGE_TYPE_2D`
  - `arrayLayers` must be 6
  - `width` must be equal to `height`

Image Tiling

- `VK_IMAGE_TILING_LINEAR` : texels are laid out in memory in row-major order, possibly with some padding on each row
- `VK_IMAGE_TILING_OPTIMAL` : texels are laid out in an implementation defined arrangement for efficient memory access
- Linear tiling is only guaranteed to be supported for 2D, non-depth/stencil images without mip-maps and array layers, and only for `TRANSFER_SRC` and `TRANSFER_DST` usage.
- Optimal tiling is much faster for sampling and rendering.
- Optimal tiled images cannot be accessed by the CPU directly.
- In practice: Use optimal tiling for every image.
  Linear tiling should (almost) never be used.
  Buffers are usually the better linear images.



Z-order curves are one possible optimal image tiling.
David Eppstein, CC BY-SA 3.0

Image Formats

```
typedef enum VkFormat {
    VK_FORMAT_UNDEFINED = 0,
    ...
    VK_FORMAT_R8_UNORM = 9,
    ...
    VK_FORMAT_R8G8_UNORM = 16,
    ...
    VK_FORMAT_R8G8B8_UNORM = 23,
    ...
    VK_FORMAT_R8G8B8A8_UNORM = 37,
    VK_FORMAT_R8G8B8A8_SNORM = 38,
    VK_FORMAT_R8G8B8A8_USCALED = 39,
    VK_FORMAT_R8G8B8A8_SSCALED = 40,
    VK_FORMAT_R8G8B8A8_UINT = 41,
    VK_FORMAT_R8G8B8A8_SINT = 42,
    VK_FORMAT_R8G8B8A8_SRGB = 43,
    ...
    VK_FORMAT_R16G16_SFLOAT = 83,
    ...
} VkFormat;
```

- Components and bit counts in memory order: `R8G8B8A8` = four 8-bit components
- `UNORM` : unsigned normalized in [0, 1]
- `SNORM` : signed normalized in [-1, 1]
- `USCALED` : unsigned integers read as float
- `SSCALED` : signed integers read as float
- `UINT` : unsigned integers
- `SINT` : signed integers
- `SFLOAT` : signed floating-point numbers
- `SRGB` : unsigned normalized values in nonlinear sRGB encoding

Texels of color formats must be represented in memory exactly as in the format’s name.

Image Formats

```
typedef enum VkFormat {
    ...
    VK_FORMAT_R5G6B5_UNORM_PACK16 = 4,
    VK_FORMAT_B5G6R5_UNORM_PACK16 = 5,
    VK_FORMAT_R5G5B5A1_UNORM_PACK16 = 6,
    VK_FORMAT_B5G5R5A1_UNORM_PACK16 = 7,
    ...
    VK_FORMAT_A2R10G10B10_UNORM_PACK32 = 58,
    ...
    VK_FORMAT_B10G11R11_UFLOAT_PACK32 = 122,
    VK_FORMAT_E5B9G9R9_UFLOAT_PACK32 = 123,
    ...
    VK_FORMAT_D16_UNORM = 124,
    VK_FORMAT_X8_D24_UNORM_PACK32 = 125,
    VK_FORMAT_D32_SFLOAT = 126,
    VK_FORMAT_S8_UINT = 127,
    VK_FORMAT_D16_UNORM_S8_UINT = 128,
    VK_FORMAT_D24_UNORM_S8_UINT = 129,
    ...
} VkFormat;
```

- `_PACKnn` : components packed into an underlying type with `nn` bits

Depth/stencil formats:

- `Dn` : depth component with `n` bits
- `Sn` : stencil component with `n` bits

Depth/stencil formats **may** be stored with arrangement and bits per texel different to the format. They **must** use the precision described by the format.

Image Formats

```
typedef enum VkFormat {
    ...
    VK_FORMAT_BC1_RGB_UNORM_BLOCK = 131,
    VK_FORMAT_BC1_RGB_SRGB_BLOCK = 132,
    ...
    VK_FORMAT_BC3_UNORM_BLOCK = 137,
    VK_FORMAT_BC3_SRGB_BLOCK = 138,
    ...
    VK_FORMAT_BC7_UNORM_BLOCK = 145,
    VK_FORMAT_BC7_SRGB_BLOCK = 146,
    ...
    VK_FORMAT_ETC2_R8G8B8A8_SRGB_BLOCK = 152,
    VK_FORMAT_EAC_R11_UNORM_BLOCK = 153,
    ...
    VK_FORMAT_ASTC_4x4_UNORM_BLOCK = 157,
    VK_FORMAT_ASTC_4x4_SRGB_BLOCK = 158,
    ...
    VK_FORMAT_ASTC_12x12_SRGB_BLOCK = 184,
    ...
} VkFormat;
```

Block-Compressed Formats

Desktop formats:

- BC1, BC2, BC3 (S3TC)
  → RGB / RGBA (low to medium quality)
- BC4, BC5 (RGTC)
  → Grayscale
- BC6, BC7 (BPTC)
  → RGB / RGBA (high quality)

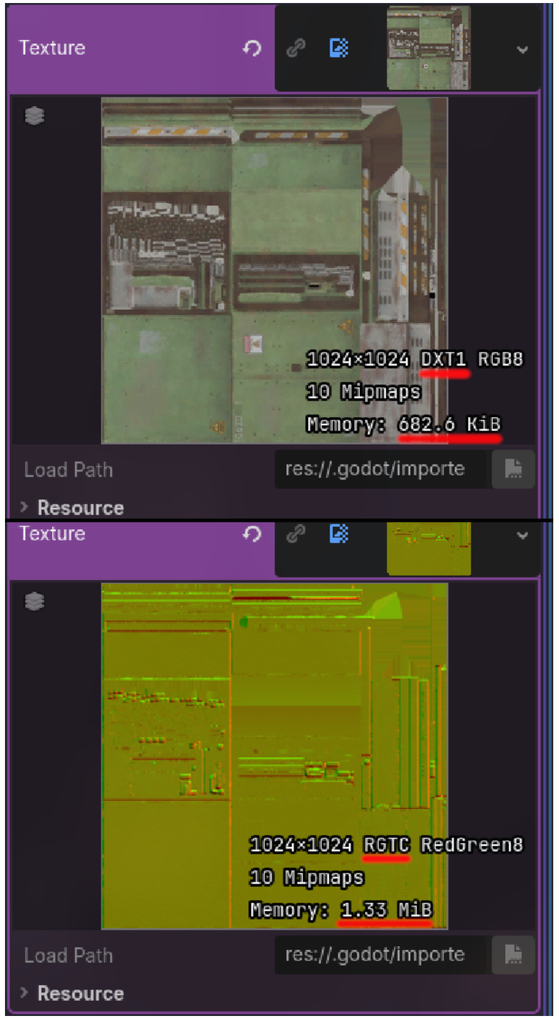
Mobile formats:

- ETC2 / EAC
  → similar to S3TC / RGTC
- ASTC
  → similar to BPTC

Block-Compressed Image Formats

- Uncompressed textures on the GPU use lots of memory
- Most textures have little (local) color variation
- It seems intuitive to store the textures compressed
- Important properties:
  - Random access
  - High decompression speed
  - High compression ratio
  - Acceptable visual quality

→ Block-compressed lossy image formats



Texture compression in Godot

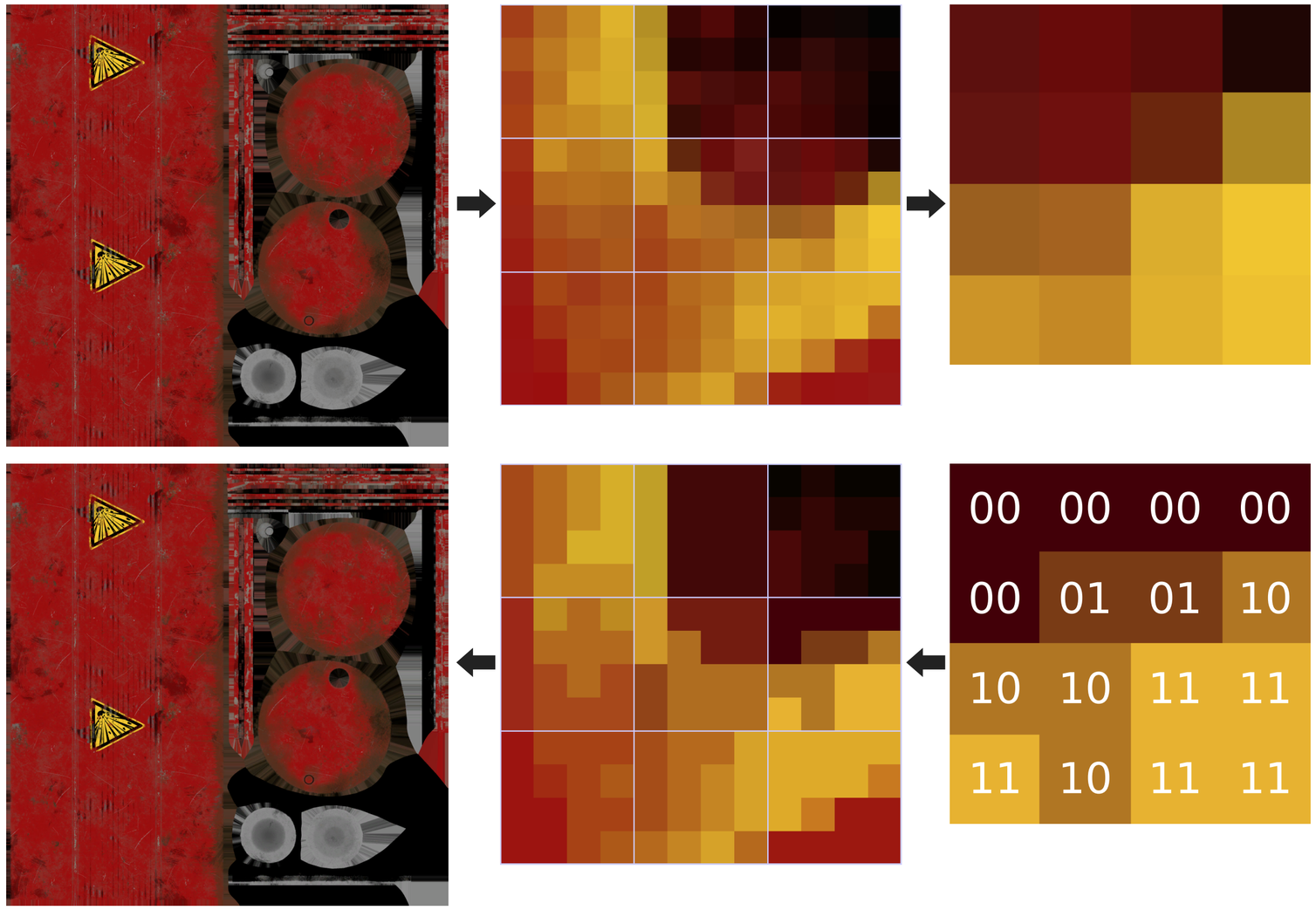
Block-Compressed Formats Coverage

Windows	
BC1_RGB_SRGB_BLOCK	99.81%
BC1_RGB_UNORM_BLOCK	99.81%
ASTC_10x10_SRGB_BLOCK	7.56%
ASTC_10x10_UNORM_BLOCK	7.56%
ETC2_R8G8B8_SRGB_BLOCK	9.11%
ETC2_R8G8B8_UNORM_BLOCK	9.11%

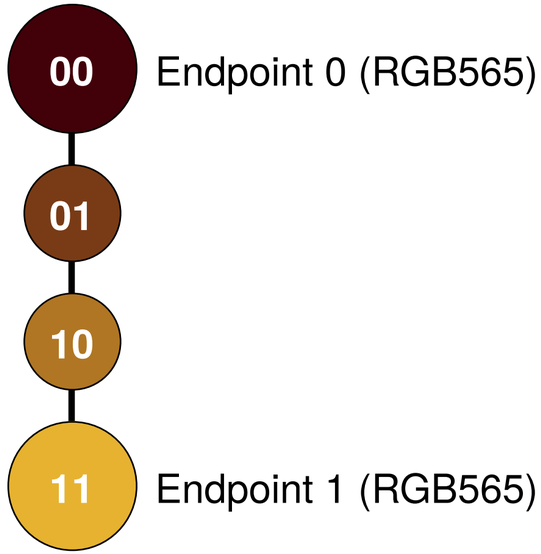
Android	
BC1_RGB_SRGB_BLOCK	3.91%
BC1_RGB_UNORM_BLOCK	3.91%
ASTC_10x10_SRGB_BLOCK	97.53%
ASTC_10x10_UNORM_BLOCK	97.53%
ETC2_R8G8B8_SRGB_BLOCK	99.48%
ETC2_R8G8B8_UNORM_BLOCK	99.48%

From: Vulkanised texture compression talk - Sascha Willems

BC1 (DXT1)

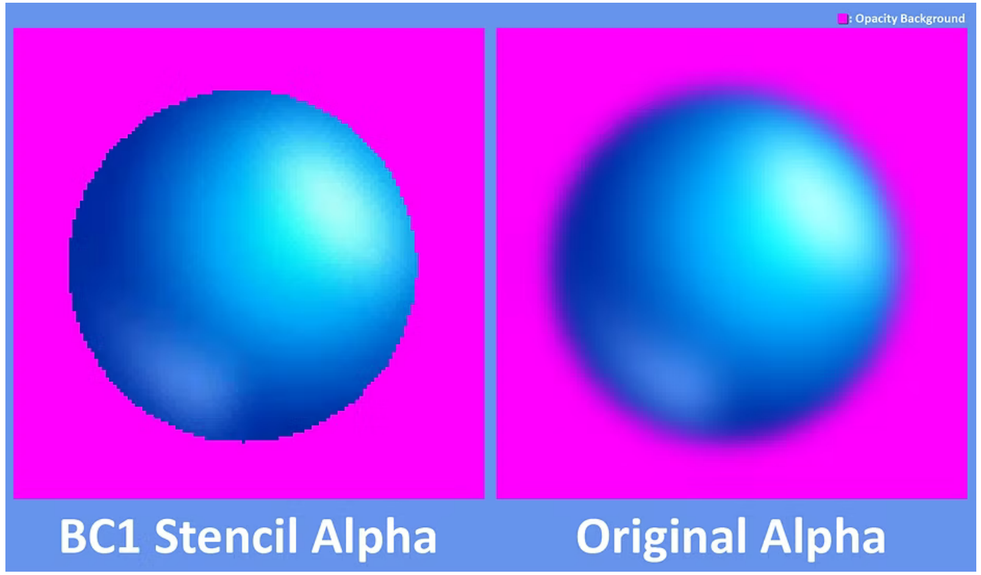


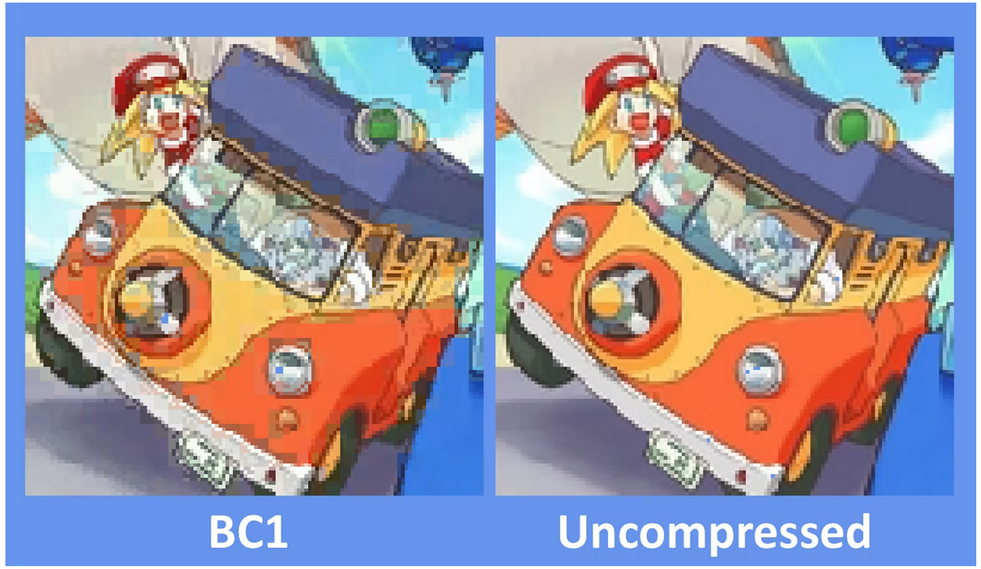
- Divide in 4x4 texel blocks
- For each block store two endpoint colors in RGB565
- Store two bits per texel to blend between endpoints (0%, 33%, 66%, 100%)
- Total: 8 bytes per block



BC1 (DXT1)

- Requires only 0.5 bytes per texel
  (1/6 of original image for RGB and 1/8 for RGBA)
- RGB565 struggles with smooth gradients
- Supports 1-bit alpha:
  - Enabled when endpoint 0 <= endpoint 1
  - Then, 11 = transparent
  - Reduces block to three colors
- ‘Color clashing’ common on grid boundary
  → ‘Stair stepping’ on diagonal color boundaries





https://www.connburanicz.com/block-compression-bc1-bc7

BCn

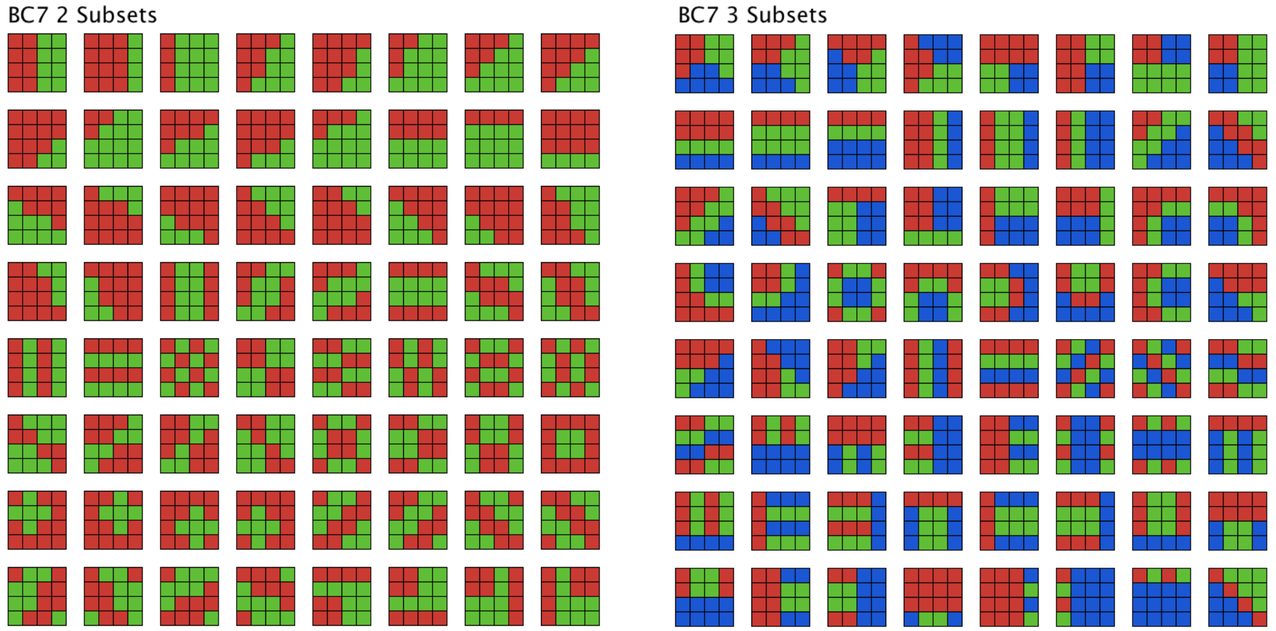
- **BC2** (DXT2/3) : BC1 + 4bit per texel alpha value
- **BC3** (DXT4/5) : BC1 + separately encoded alpha channel
  - Two one byte endpoints per tile
  - 3bit index per texel to interpolate between endpoints
  - Alternative mode where two indices map to 0 and 1
- **BC4** (RGTC1) : only the BC3 alpha channel
- **BC5** (RGTC2) : two BC3 alpha channels
- **BC6H** and **BC7** (BPTC)
  - Very complex modern scheme for high-quality textures
  - Use different modes and partition schemes per tile

Normal Map Compression (3Dc™ White Paper)

Uncompressed **BC1** **BC5**

BC7

- 16 bytes per block (stores mode, partition, rotation, index selector, endpoints, indices)
- Each block uses one of eight modes (4 opaque modes, 4 transparent modes)
- Partition: some modes split tile into two or three subsets with individual endpoints
- 2-, 3- or 4-bit indices indices to interpolate between endpoints
- Endpoint precision depends on mode



Visualization by Jon Rocatis

BC7

- BC7 has very high quality, nearly indistinguishable from un-compressed textures
- Uses the same data rate as BC3
- Nearly same decoding speed, as all BCn formats are typically implemented in hardware
- Very compute-intensive encoding, usually performed once during import of asset in engine

https://www.connburanicz.com/block-compression-bc1-bc7

BCn usage

	Texel Data	Data Rate	Usage
BC1	RGB (+opt. 1bit alpha)	0.5 byte / px	standard color textures without alpha
BC2	RGB + 4bit alpha	1 byte / px	-
BC3	RGBA	1 byte / px	standard color textures with alpha
BC4	R	0.5 byte / px	roughness, height maps, masks
BC5	RG	1 byte / px	tangent-space normal maps
BC6	RGB (float)	1 byte / px	HDR images without alpha
BC7	RGB or RGBA	1 byte / px	high quality color maps (with alpha)

Common Image Formats

- Use available block-compressed formats for static textures
- SDR render targets: `VK_FORMAT_R8G8B8A8_SRGB` (sometimes `VK_FORMAT_R8G8B8A8_UNORM`)
- HDR render targets: `B10G11R11_UFLOAT_PACK32` or `VK_FORMAT_R16G16B16A16_SFLOAT` or `VK_FORMAT_R32G32B32A32_SFLOAT` (without alpha is rarely supported)
- For storing normals: `VK_FORMAT_A2B10G10R10_UNORM_PACK32`
- Depth formats:
  - Usually `VK_FORMAT_D32_SFLOAT` for depth buffer (if supported)
  - Shadow maps: `VK_FORMAT_D16_UNORM`
  - With stencil: `VK_FORMAT_D24_UNORM_S8_UINT` (if supported) or `VK_FORMAT_D32_SFLOAT_S8_UINT`

Format Support

Format	VK_FORMAT_FEATURE_SAMPLED_IMAGE_BIT	VK_FORMAT_FEATURE_BLIT_SRC_BIT	VK_FORMAT_FEATURE_SAMPLED_IMAGE_FILTER_LINEAR_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_BIT	VK_FORMAT_FEATURE_STORAGE_IMAGE_ATOMIC_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BIT	VK_FORMAT_FEATURE_BLIT_DST_BIT	VK_FORMAT_FEATURE_COLOR_ATTACHMENT_BLEND_BIT	VK_FORMAT_FEATURE_DEPTH_STENCIL_ATTACHMENT_BIT
VK_FORMAT_D16_UNORM	✓	✓							✓
VK_FORMAT_X8_D24_UNORM_PACK32									†
VK_FORMAT_D32_SFLOAT	✓	✓							†
VK_FORMAT_S8_UINT									
VK_FORMAT_D16_UNORM_S8_UINT									
VK_FORMAT_D24_UNORM_S8_UINT									†
VK_FORMAT_D32_SFLOAT_S8_UINT									†

- The Vulkan specification defines a list of required formats and features.
- Example:
  From the depth formats, only `VK_FORMAT_D16_UNORM` and `VK_FORMAT_D32_SFLOAT` are required to allow shader reading.
- Every other usage must be queried with `vkGetPhysicalDeviceFormatProperties`, which returns:

```
1 typedef struct VkFormatProperties {
2     VkFormatFeatureFlags    linearTilingFeatures;
3     VkFormatFeatureFlags    optimalTilingFeatures;
4     VkFormatFeatureFlags    bufferFeatures;
5 } VkFormatProperties;
```

Image Usage

```
    VkImageUsageFlags                usage;
```

```
typedef enum VkImageUsageFlagBits {
    VK_IMAGE_USAGE_TRANSFER_SRC_BIT = 0x00000001,
    VK_IMAGE_USAGE_TRANSFER_DST_BIT = 0x00000002,
    VK_IMAGE_USAGE_SAMPLED_BIT = 0x00000004,
    VK_IMAGE_USAGE_STORAGE_BIT = 0x00000008,
    VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT = 0x00000010,
    VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT = 0x00000020,
    VK_IMAGE_USAGE_TRANSIENT_ATTACHMENT_BIT = 0x00000040,
    VK_IMAGE_USAGE_INPUT_ATTACHMENT_BIT = 0x00000080,
    ...
} VkImageUsageFlagBits;
```

- `TRANSFER_SRC` / `TRANSFER_DST`: source or destination of copy and blit commands
- `SAMPLED`: read through a sampler in shaders
- `STORAGE`: read and write with `imageLoad` / `imageStore` in shaders
- `COLOR_ATTACHMENT` / `DEPTH_STENCIL_ATTACHMENT`: render targets
- Only necessary flags should be specified. Any additional flags can prevent driver optimizations (e.g. compression).

Image Memory

```
1  const VkImageCreateInfo imageInfo{
2      .sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO,
3      ...
4  };
5  VK_CHECK(vkCreateImage(device, &imageInfo, nullptr, &image));
6
7  VkMemoryRequirements memRequirements;
8  vkGetImageMemoryRequirements(device, image, &memRequirements);
9
10 // find first memory type that has its bit set in memoryTypeBits and has the required properties
11 const uint32_t memoryTypeIndex = findMemoryType(memRequirements.memoryTypeBits, propertyFlags);
12
13 const VkMemoryAllocateInfo allocInfo{
14     .sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO,
15     .allocationSize = memRequirements.size,
16     .memoryTypeIndex = memoryTypeIndex,
17 };
18 VK_CHECK(vkAllocateMemory(device, &allocInfo, nullptr, &memory));
19
20 VK_CHECK(vkBindImageMemory(device, image, memory, 0));
```

Image Memory Requirements

- Image memory usually has higher alignment requirements than buffer memory.
- Images usually have a minimum total size and minimum row count.
- Optimal tiled images can typically only be placed in `DEVICE_LOCAL` memory.
- Buffers and images must respect `bufferImageGranularity`:
  - Linear resources (buffers, linear images) and non-linear resources (optimal images) must not be allocated from the same ‘page’ of this size.
  - Typically between 1 byte and 64 kB on most desktop GPUs
  - VMA handles this automatically

```
const VmaAllocationCreateInfo allocCreateInfo{
    .usage = VMA_MEMORY_USAGE_AUTO,
};

VkImage image;
VmaAllocation allocation;
vmaCreateImage(allocator, &imageInfo, &allocCreateInfo, &image, &allocation, nullptr);
```

Image Layouts

```
typedef enum VkImageLayout {
    VK_IMAGE_LAYOUT_UNDEFINED = 0,
    VK_IMAGE_LAYOUT_GENERAL = 1,
    VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL = 2,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL = 3,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL = 4,
    VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL = 5,
    VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL = 6,
    VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL = 7,
    VK_IMAGE_LAYOUT_PREINITIALIZED = 8,
    // Provided by VK_KHR_swapchain
    VK_IMAGE_LAYOUT_PRESENT_SRC_KHR = 1000001002,
    ...
} VkImageLayout;
```

- The internal storage of optimally tiled images is undefined.
- The best arrangement can change based on how the image is used.
- To help the driver choose optimal arrangements, images are always in a certain layout.
- Layout transitions change the layout, which can trigger rearrangement of the data.

Image Layouts

```
typedef enum VkImageLayout {
    VK_IMAGE_LAYOUT_UNDEFINED = 0,
    VK_IMAGE_LAYOUT_GENERAL = 1,
    VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL = 2,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL = 3,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_READ_ONLY_OPTIMAL = 4,
    VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL = 5,
    VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL = 6,
    VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL = 7,
    VK_IMAGE_LAYOUT_PREINITIALIZED = 8,
    // Provided by VK_KHR_swapchain
    VK_IMAGE_LAYOUT_PRESENT_SRC_KHR = 1000001002,
    ...
} VkImageLayout;
```

- `UNDEFINED`: contents are undefined. Can be used as initial layout or when transitioning to discard previous data.
- `GENERAL`: supports all types of device access. Potentially slower. Required for storage images.
- `*_OPTIMAL`: allows only one kind of access, but is most optimized for that.
- `PREINITIALIZED`: for linear images written by the host before first use.

Why Image Layouts?

Fast Clears

- Resetting an entire framebuffer to a default values means writing lots of memory.
- Solution: Save an additional bit-plane that stores if the texel/block is cleared.
- Combining both planes and the clear color may be too complex for texture units.
- Instead: Transform to a ‘normal’ image before reading.
- Driver knows when to do this: on transition
  `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`
  → `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`.

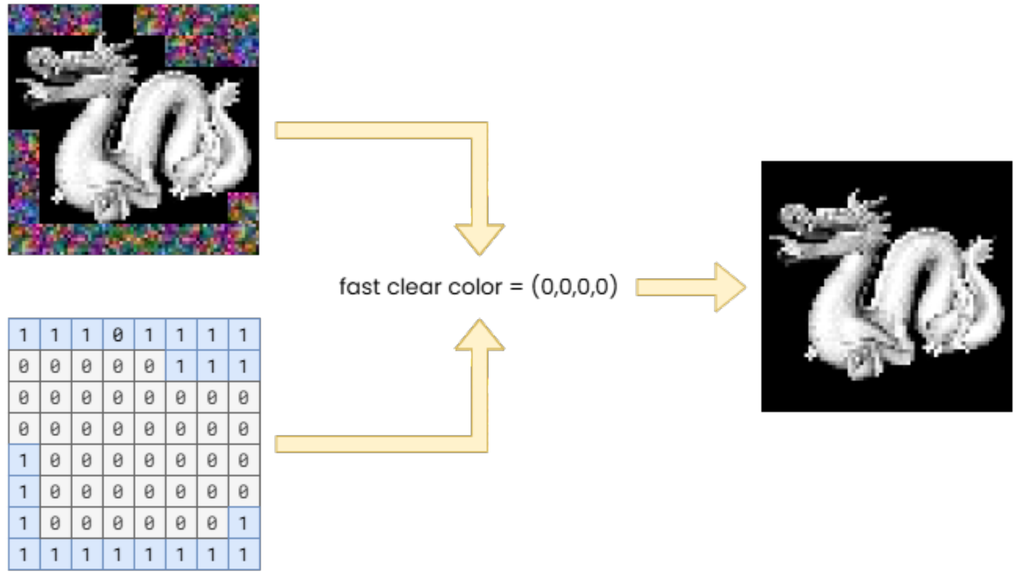


Image by RasterGrid

Why Image Layouts?

Delta Color Compression

- Lossless compression scheme for render targets.
- Typically used for non-multisampled color images.
- GPU stores blocks containing a set of reference pixels and delta values from that reference.
- Compression ratio varies over the image.
- Massively reduces usage of memory bandwidth.
- Separate meta-plane to store how blocks are compressed.

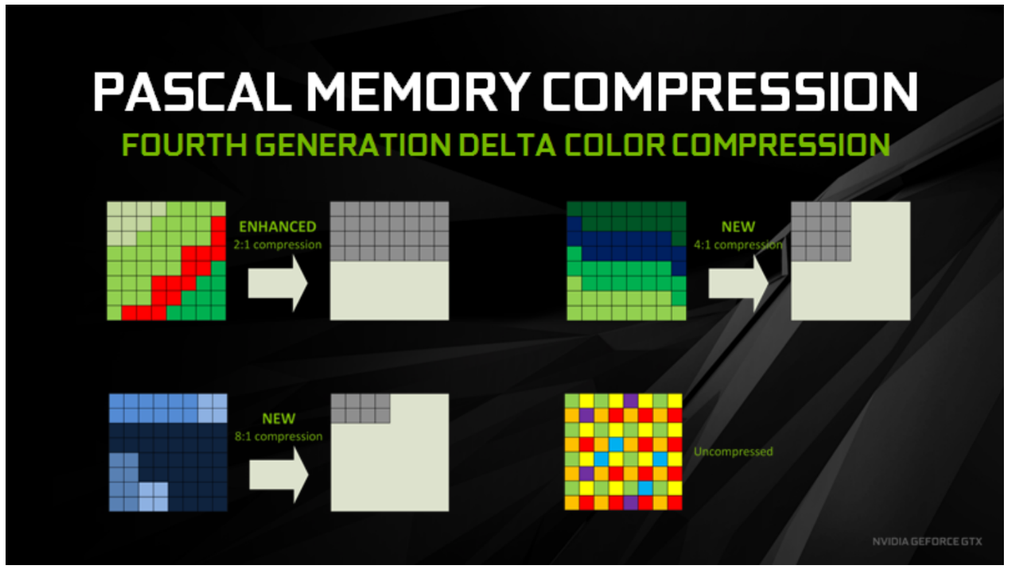


Image from NVIDIA GeForce GTX 1080 Whitepaper

- Some GPUs might choose to decompress these images before giving shaders access.
- A layout transition helps the driver determine when the image is no longer used as a render target and what comes next.

Why Image Layouts?

Hierarchical Depth Buffer

- Store a low-resolution plane along with the depth buffer that stores min. and/or max. values.
- Speeds up depth tests if fragment is completely in front/back of the existing geometry.
- Transitioning from `DEPTH_STENCIL_ATTACHMENT_OPTIMAL` is usually a no-op, as the image still contains the full depth map (although it might be compressed).
- Transitioning to `DEPTH_STENCIL_ATTACHMENT_OPTIMAL` can cause a re-build of the low-resolution plane, as the depth values might have changed.

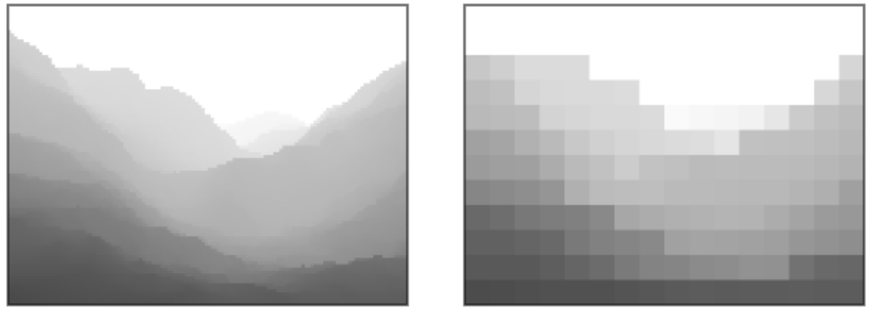


Image by RasterGrid

Image Layouts

- Optimal tiled images can be compressed, store additional meta-planes, or use different texel arrangements depending on their current usage.
- Explicit layout transitions help the driver optimize conversions between representations.
- Layouts are tracked per subresource. Different mip levels and array layers can be in different layouts.
- Transitions are performed
  - automatically by render passes (next lecture)
  - manually through memory barriers (this lecture)

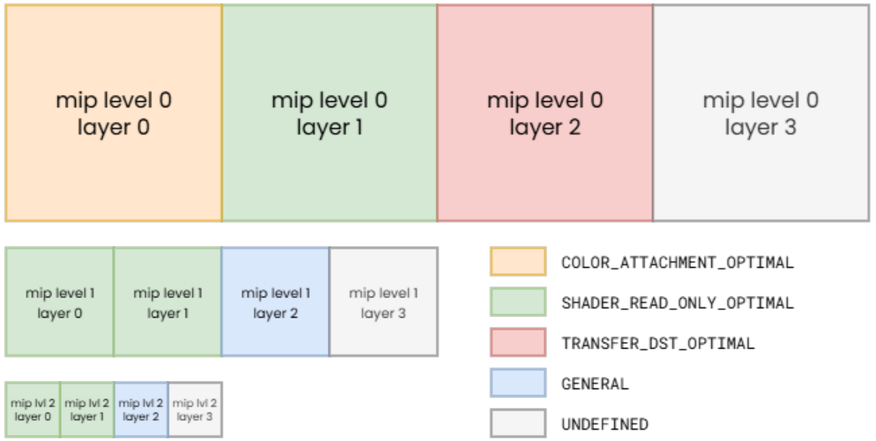


Image by RasterGrid

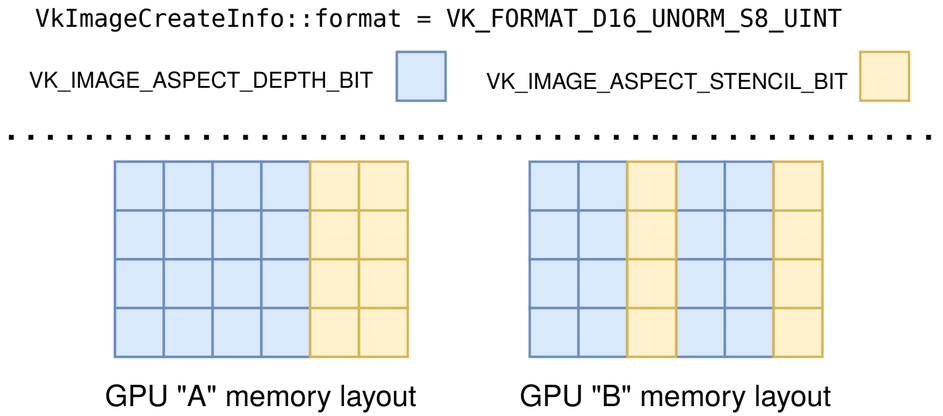
Image Subresources

```
typedef struct VkImageSubresourceRange {
    VkImageAspectFlags    aspectMask;
    uint32_t              baseMipLevel;
    uint32_t              levelCount;
    uint32_t              baseArrayLayer;
    uint32_t              layerCount;
} VkImageSubresourceRange;

typedef enum VkImageAspectFlagBits {
    VK_IMAGE_ASPECT_COLOR_BIT = 0x00000001,
    VK_IMAGE_ASPECT_DEPTH_BIT = 0x00000002,
    VK_IMAGE_ASPECT_STENCIL_BIT = 0x00000004,
    VK_IMAGE_ASPECT_METADATA_BIT = 0x00000008,
} VkImageAspectFlagBits;
```

- Depth/stencil formats have two aspects.
- Their relative layout is not defined.
- They can be accessed through subresources.

- A subresource is one mip level of one array layer of one aspect.
- Subresource ranges select one or multiple subresources.
- `VK_REMAINING_MIP_LEVELS` and `VK_REMAINING_ARRAY_LAYERS` select everything from the base onward.



https://docs.vulkan.org/guide/latest/image_copies.html

Image Views

```
1  VkResult vkCreateImageView(
2      VkDevice                            device,
3      const VkImageViewCreateInfo*        pCreateInfo,
4      const VkAllocationCallbacks*        pAllocator,
5      VkImageView*                        pView);
```

```
1  typedef struct VkImageViewCreateInfo {
2      VkStructureType                     sType;
3      const void*                         pNext;
4      VkImageViewCreateFlags              flags;
5      VkImage                             image;
6      VkImageViewType                     viewType;
7      VkFormat                            format;
8      VkComponentMapping                  components;
9      VkImageSubresourceRange             subresourceRange;
10 } VkImageViewCreateInfo;
```

- Images are never used directly by shaders or as attachments.
- Image views describe how and which part of the image to access.
- `format` must match the image format (unless created with `VK_IMAGE_CREATE_MUTABLE_FORMAT_BIT` with compatible formats, e.g. converting SRGB → linear)

Image Views

```
1 typedef enum VkImageViewType {
2     VK_IMAGE_VIEW_TYPE_1D = 0,
3     VK_IMAGE_VIEW_TYPE_2D = 1,
4     VK_IMAGE_VIEW_TYPE_3D = 2,
5     VK_IMAGE_VIEW_TYPE_CUBE = 3,
6     VK_IMAGE_VIEW_TYPE_1D_ARRAY = 4,
7     VK_IMAGE_VIEW_TYPE_2D_ARRAY = 5,
8     VK_IMAGE_VIEW_TYPE_CUBE_ARRAY = 6,
9 } VkImageViewType;
```

- Image view type must match image type.
- Exceptions:
  - 1D/2D arrays viewed as 1D/2D images
  - 2D images with 6 layers and created with `VK_IMAGE_CREATE_CUBE_COMPATIBLE_BIT` viewed as cube maps
  - 3D images created with `VK_IMAGE_CREATE_2D_ARRAY_COMPATIBLE_BIT` viewed as 2D images or 2D arrays

Image Views

```
typedef struct VkComponentMapping {
    VkComponentSwizzle r;
    VkComponentSwizzle g;
    VkComponentSwizzle b;
    VkComponentSwizzle a;
} VkComponentMapping;

typedef enum VkComponentSwizzle {
    VK_COMPONENT_SWIZZLE_IDENTITY = 0,
    VK_COMPONENT_SWIZZLE_ZERO = 1,
    VK_COMPONENT_SWIZZLE_ONE = 2,
    VK_COMPONENT_SWIZZLE_R = 3,
    VK_COMPONENT_SWIZZLE_G = 4,
    VK_COMPONENT_SWIZZLE_B = 5,
    VK_COMPONENT_SWIZZLE_A = 6,
} VkComponentSwizzle;
```

- Remapping of components when **reading** through the image view.
- Examples:
  - read single channel `R8_UNORM` as greyscale
  - read RGB image as BRG in shader

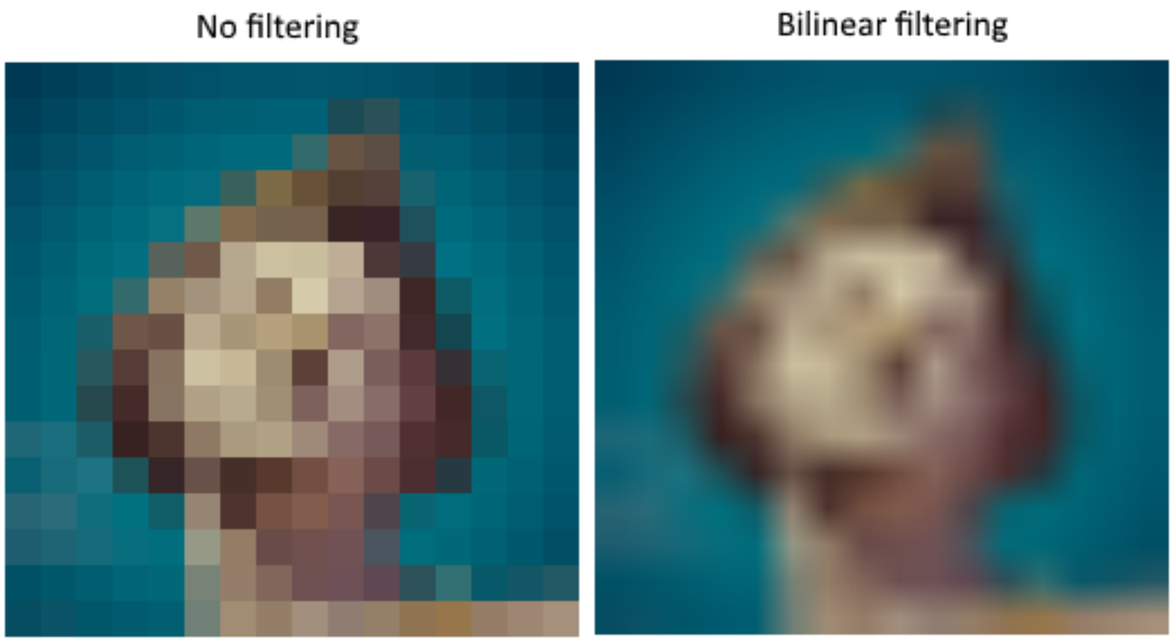
Samplers

```
typedef struct VkSamplerCreateInfo {
    VkStructureType         sType;
    const void*             pNext;
    VkSamplerCreateFlags    flags;
    VkFilter                magFilter;
    VkFilter                minFilter;
    VkSamplerMipmapMode     mipmapMode;
    VkSamplerAddressMode    addressModeU;
    VkSamplerAddressMode    addressModeV;
    VkSamplerAddressMode    addressModeW;
    float                   mipLodBias;
    VkBool32                anisotropyEnable;
    float                   maxAnisotropy;
    VkBool32                compareEnable;
    VkCompareOp             compareOp;
    float                   minLod;
    float                   maxLod;
    VkBorderColor           borderColor;
    VkBool32                unnormalizedCoordinates;
} VkSamplerCreateInfo;
```

- Created with `vkCreateSampler`.
- Describe how an image is read.
- One sampler can be used for many images.
- Number of samplers is limited: `maxSamplerAllocationCount` is guaranteed to be at least 4000 (many GPUs don’t support more).
- Typical applications need only a handful of samplers.

Samplers - Filtering

```
1       VkFilter                magFilter;
2       VkFilter                minFilter;
```



Vulkan Tutorial - Image view and sampler

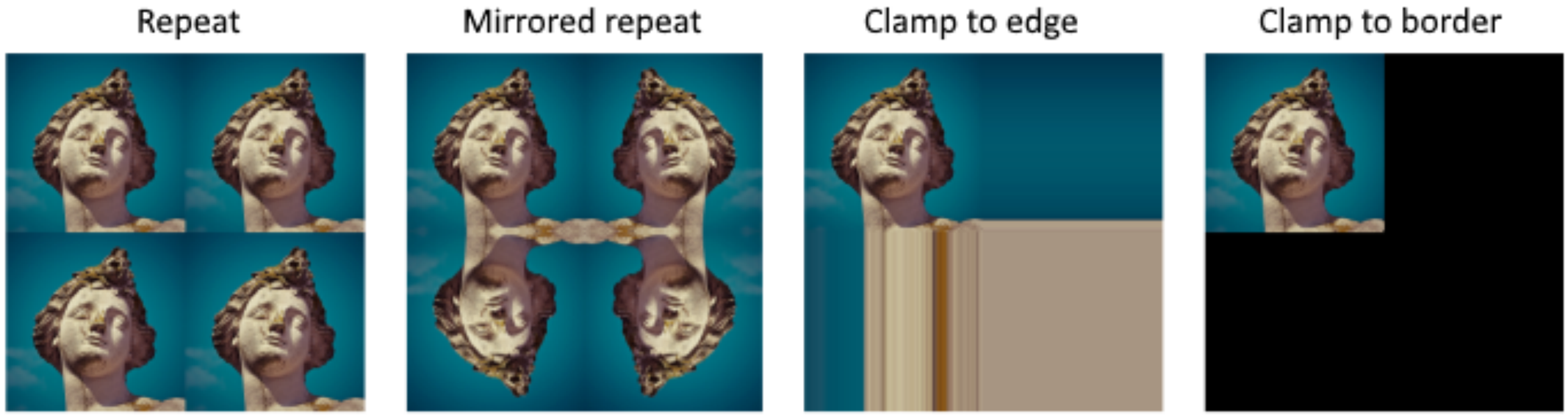
- Controls how texels that are magnified or minified are interpolated.
- `VK_FILTER_NEAREST` samples only the nearest texel
- `VK_FILTER_LINEAR` takes 4 samples for 2D images
- `VK_FILTER_CUBIC_EXT` takes 16 samples for 2D images (requires extension)

Samplers - Address Mode

```
1     VkSamplerAddressMode    addressModeU;
2     VkSamplerAddressMode    addressModeV;
3     VkSamplerAddressMode    addressModeW;
```

- Specifies behavior when texture coordinates are outside of image.

```
1 typedef enum VkSamplerAddressMode {
2     VK_SAMPLER_ADDRESS_MODE_REPEAT = 0,
3     VK_SAMPLER_ADDRESS_MODE_MIRRORED_REPEAT = 1,
4     VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_EDGE = 2,
5     VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER = 3,
6     ...
7 } VkSamplerAddressMode;
```



Vulkan Tutorial - Image view and sampler

Samplers - Border Color

```
    VkBorderColor           borderColor;
```

```
typedef enum VkBorderColor {
    VK_BORDER_COLOR_FLOAT_TRANSPARENT_BLACK = 0,
    VK_BORDER_COLOR_INT_TRANSPARENT_BLACK = 1,
    VK_BORDER_COLOR_FLOAT_OPAQUE_BLACK = 2,
    VK_BORDER_COLOR_INT_OPAQUE_BLACK = 3,
    VK_BORDER_COLOR_FLOAT_OPAQUE_WHITE = 4,
    VK_BORDER_COLOR_INT_OPAQUE_WHITE = 5,
    // Provided by VK_EXT_custom_border_color
    VK_BORDER_COLOR_FLOAT_CUSTOM_EXT = 1000287003,
    // Provided by VK_EXT_custom_border_color
    VK_BORDER_COLOR_INT_CUSTOM_EXT = 1000287004,
} VkBorderColor;
```

- Replacement color when address mode `VK_SAMPLER_ADDRESS_MODE_CLAMP_TO_BORDER` is used
- Transparent black: [0,0,0,0]
- Opaque black: [0,0,0,1]
- Opaque white: [1,1,1,1]
- Custom colors require extensions and may be rounded.

Samplers - Mip Parameters

```
1      VkSamplerMipmapMode      mipmapMode;
2      ...
3      float                    mipLodBias;
4      VkBool32                 anisotropyEnable;
5      float                    maxAnisotropy;
6      ...
7      float                    minLod;
8      float                    maxLod;
```

```
1 typedef enum VkSamplerMipmapMode {
2     VK_SAMPLER_MIPMAP_MODE_NEAREST = 0,
3     VK_SAMPLER_MIPMAP_MODE_LINEAR = 1,
4 } VkSamplerMipmapMode;
```

- `mipmapMode` : sample from one mip-level with `VK_SAMPLER_MIPMAP_MODE_NEAREST` or sample from two mip-levels with `VK_SAMPLER_MIPMAP_MODE_LINEAR`.
- `mipLodBias`, `minLod` and `maxLod` affect mip-level selection. `maxLod` can be `VK_LOD_CLAMP_NONE`.
- `anisotropyEnable` and `maxAnisotropy` control anisotropic filtering.

Automatic Mip Selection

- Take partial derivatives of texel coordinates and their magnitudes

$$\partial s/\partial x,\ \partial t/\partial x,\ \partial r/\partial x \quad \text{and} \quad \partial s/\partial y,\ \partial t/\partial y,\ \partial r/\partial y$$
$$m_{ux} = |\partial s/\partial x| \times w_{\text{base}}, \quad m_{vx} = |\partial t/\partial x| \times h_{\text{base}}, \ \ldots$$

- Calculate scale factors

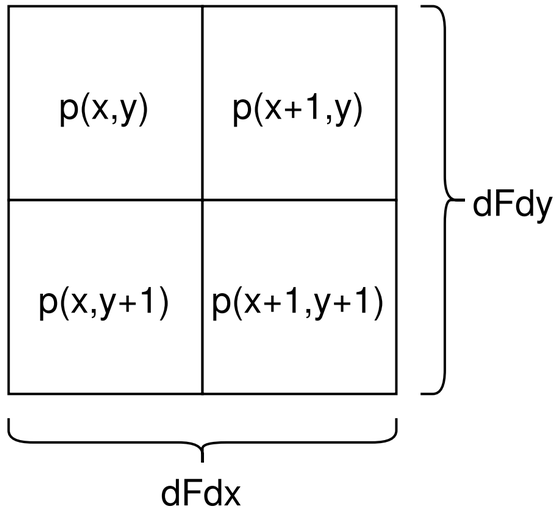
$$\rho_x = f_x(m_{ux}, m_{vx}, m_{wx}), \quad \rho_y = f_y(m_{uy}, m_{vy}, m_{wy})$$

where f is any approximation combining the magnitudes with

$$\max\left(|m_{ux}|, |m_{vx}|, |m_{wx}|\right) \leq f_x \leq \sqrt{2}\left(|m_{ux}| + |m_{vx}| + |m_{wx}|\right)$$

- Then the LOD-parameter is

$$\lambda_{base} = \log_2(\rho_{max}), \quad \text{where} \quad \rho_{max} = \max(\rho_x, \rho_y)$$
$$\lambda = \text{clamp}\left(\lambda_{base} + \text{mipLodBias}, \text{minLod}, \text{maxLod}\right)$$



GPUs rasterize in micro-quads of 2x2 pixels. Texel coordinates of `texture()` calls in neighboring pixels lead to implicit derivatives. Important: Never call `texture()` in diverging control flow. This breaks implicit derivatives.

Anisotropic Filtering

Fallout 4 Graphics, Performance & Tweaking Guide - NVIDIA

- Axis with large gradient dominates mip level selection.

$$\lambda_{base} = \log_2(\max(\rho_x, \rho_y))$$

- Ratio of anisotropy

$$\eta = \min\left(\rho_{max}/\rho_{min},\ \text{maxAniso}\right)$$

- New LOD parameter

$$\lambda_{base} = \log_2\left(\frac{\rho_{max}}{\eta}\right)$$

- Take $N = \lceil \eta \rceil$ samples in pixel footprint to counteract minification.

- Requires `samplerAnisotropy` feature on device creation.
- `maxSamplerAnisotropy` is upper limit for `maxAnisotropy` and is 16 on nearly all devices.

Samplers - Depth Compare Operation

```
    VkBool32                    compareEnable;
    VkCompareOp                 compareOp;
```

```
typedef enum VkCompareOp {
    VK_COMPARE_OP_NEVER = 0,
    VK_COMPARE_OP_LESS = 1,
    VK_COMPARE_OP_EQUAL = 2,
    VK_COMPARE_OP_LESS_OR_EQUAL = 3,
    VK_COMPARE_OP_GREATER = 4,
    VK_COMPARE_OP_NOT_EQUAL = 5,
    VK_COMPARE_OP_GREATER_OR_EQUAL = 6,
    VK_COMPARE_OP_ALWAYS = 7,
} VkCompareOp;
```

- Supported on depth/stencil formats.
- Used with `sampler2DShadow` in GLSL.
- Shadow samplers receive 3D coordinates.
  `xy` is the normalized texel coordinate.
  `z` is the reference depth.
- Reference depth is compared against shadow map. 1.0 is returned if comparison evaluates to true, 0.0 otherwise.
- If linear filtering is used, the result is proportional to a weighted average of the number of comparisons that pass (Percentage-Closer Filtering).

Samplers - Unnormalized Coordinates

```
1    VkBool32                 unnormalizedCoordinates;
```

- Use unnormalized coordinates when sampling images.
- The ranges are then $[0, w[$, $[0, h[$
- Restrictions:
  - The view must be 1D or 2D.
  - The view must have a single layer and single mip level.
  - `minFilter` and `magFilter` must be equal.
  - `addressMode*` must be `CLAMP_TO_BORDER/_EDGE`.
  - The sampling function must not use projection.
  - The sampling function must not use offsets.

Images in Shaders

```
1  // combined image sampler
2  layout (binding = 0) uniform sampler2D albedo;
3
4  // separate image and sampler
5  layout (binding = 1) uniform texture2D normalMap;
6  layout (binding = 2) uniform sampler linearSampler;
7
8  // storage image, format must be specified
9  layout (binding = 3, rgba8) uniform writeonly image2D outImage;
10
11 void main() {
12     ...
13
14     vec4 a = texture(albedo, uv);
15     vec4 n = texture(sampler2D(normalMap, linearSampler), uv);
16
17     ivec2 pixel = ivec2(uv * size);
18     imageStore(outImage, pixel, a * n);
19 }
```

- `texture`: filtered read using normalized coordinates
- `textureGrad`: supply explicit gradients for mip level
- `textureLod`: supply mip level directly
- `texelFetch`: unfiltered read of a single texel with integer coordinates
- `imageLoad` / `imageStore`: unfiltered read/write of storage images with integer coordinates

Image Descriptors

```
1  const VkDescriptorImageInfo textureInfo{
2      .sampler = m_sampler,
3      .imageView = m_textureView,
4      .imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL,
5  };
6
7  const VkDescriptorImageInfo storageInfo{
8      .sampler = VK_NULL_HANDLE,
9      .imageView = m_storageView,
10     .imageLayout = VK_IMAGE_LAYOUT_GENERAL,
11 };
```

- The image has to be in `imageLayout` when the shader accesses it.
- Descriptors do not change the layout of images. They have to be transitioned beforehand.

Uploading Image Data

```
1  void vkCmdCopyBufferToImage(
2      VkCommandBuffer                commandBuffer,
3      VkBuffer                       srcBuffer,
4      VkImage                        dstImage,
5      VkImageLayout                  dstImageLayout,
6      uint32_t                       regionCount,
7      const VkBufferImageCopy*       pRegions);
```

```
1  typedef struct VkBufferImageCopy {
2      VkDeviceSize                   bufferOffset;
3      uint32_t                       bufferRowLength;
4      uint32_t                       bufferImageHeight;
5      VkImageSubresourceLayers       imageSubresource;
6      VkOffset3D                     imageOffset;
7      VkExtent3D                     imageExtent;
8  } VkBufferImageCopy;
```

- Staging approach as for buffers.
- `bufferOffset` is an offset in bytes from the start of the buffer.
- `bufferRowLength` and `bufferImageHeight` specify the layout of the data in the buffer (in texels). Zero means tightly packed.
- `imageSubresource` specifies one mip level and any number of array layers.
- `dstImageLayout` must be `TRANSFER_DST_OPTIMAL` or `GENERAL`.

How do we get the image from `UNDEFINED` to `TRANSFER_DST_OPTIMAL`?

Recap - Images

- Images are multidimensional arrays of texels and have mip levels, array layers, a format, tiling and layout.
- Mipmaps are multiple levels of different sized images.
- Array layers are multiple layers of same size images.
- Use optimal tiling for all images → internal memory layout is undefined.
- Image formats define per-texel data.
- Block-compressed formats are used for textures.
- Image layouts track internal state of images. Manually transitioned by application.
- Render targets are automatically compressed by GPU. Image layouts help optimize this.
- Images are mainly accessed through image views and samplers.
- Samplers define filtering and addressing and are independent of images.

Synchronization

Asynchronous Execution

- Vulkan provides only a small number of implicit guarantees about execution order.
  - Commands in command buffers start in order, but may overlap and complete in any order.
  - Command buffers submitted to the same queue start in order, but may also overlap.
  - Different queues are completely independent from each other.
  - The CPU runs completely independent of the GPU.

⇒ Any synchronization and dependencies are explicit.

Hazards

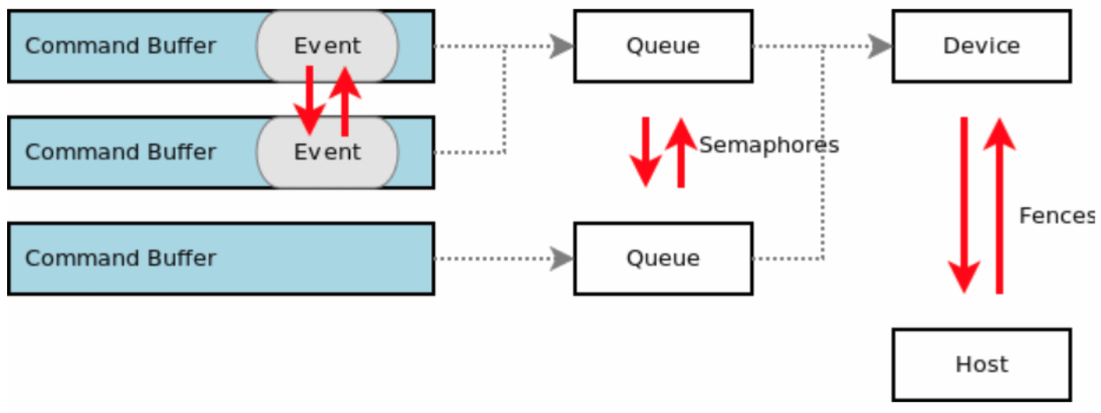
- **Read-after-write (RAW)**
  A dispatch reads a buffer written by the previous dispatch.

- **Write-after-read (WAR)**
  A copy overwrites an image that a shader is still reading.

- **Write-after-write (WAW)**
  Two dispatches write to the same image, the second result must be the final one.

- **Read-after-read**
  Not a hazard, no synchronization needed.

Synchronization Primitives



https://docs.vulkan.org/guide/latest/synchronization.html

- **Fences**: Signal to the host that a GPU task has completed.
- **Semaphores**: Synchronize across queues, but also between submissions on the same queue and with the presentation engine.
- **Events**: Fine grained synchronization within a single queue.
- **Pipeline Barriers**: Synchronize resource access within a single queue.
- **Render Passes**: Provide synchronization framework for standard rendering tasks.
- **Wait idle**: Blocks the CPU until all work is done (`vkQueueWaitIdle`/`vkDeviceWaitIdle`).

Execution vs. Memory Dependencies

- An **Execution Dependency** answers the question: “When can this work start?”
- An execution dependency would prevent Command B to start before Command A has finished.
- *Is an execution dependency enough to prevent a read-after-write hazard between Command A and Command B?*
- **No.** Command A may have finished writing the data, but it may still sit in a cache, inaccessible to Command B.

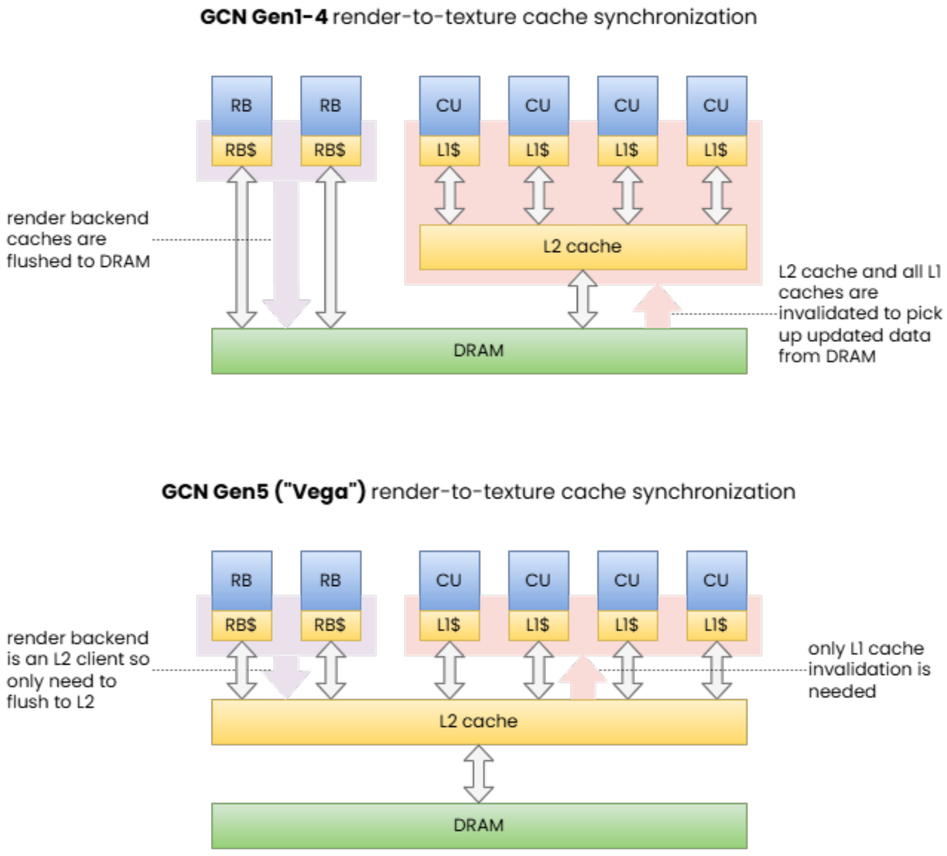


Image by RasterGrid

Execution vs. Memory Dependencies

- GPUs have many caches. Writes to caches may not propagate automatically.
- A **memory dependency** extends an execution dependency with two steps:
  - **Availability**: the data is flushed from the source's local cache to a shared memory pool (like L2 cache or main memory).
  - **Visibility**: the local cache of the destination is invalidated, forcing a fresh read from the shared memory pool.

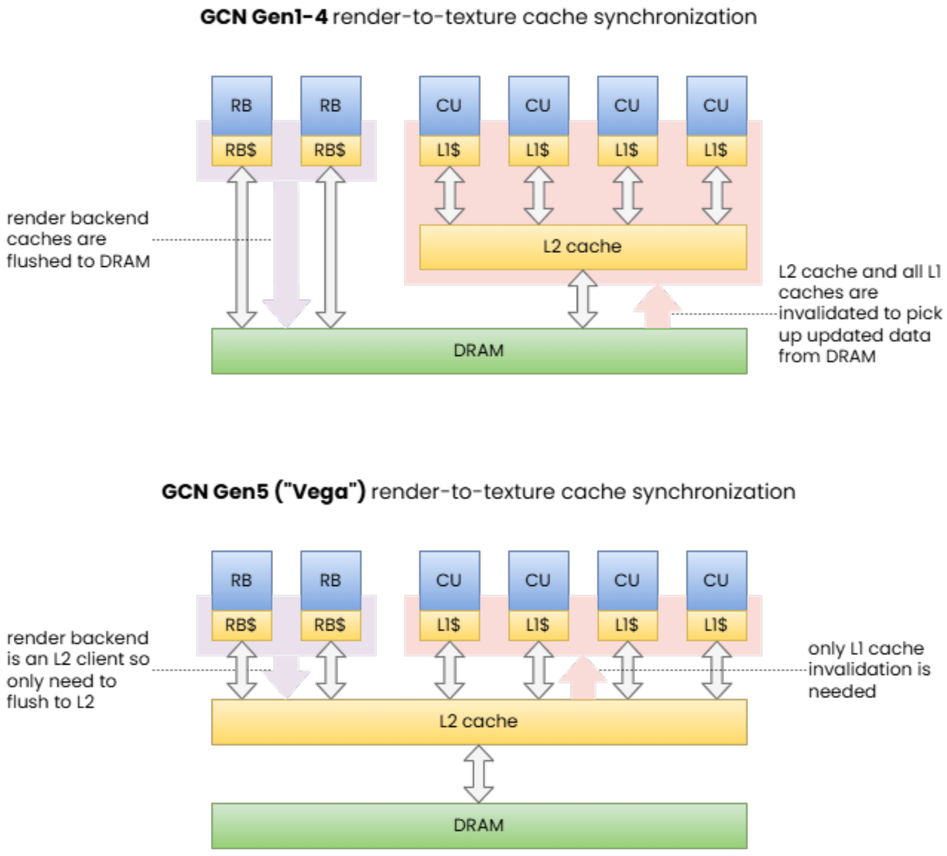


Image by RasterGrid

Example: Two Compute Shaders

```
1  void recordCommandBuffer(VkCommandBuffer commandBuffer) {
2      ...
3      vkCmdDispatch(commandBuffer, NUM_WORKGROUPS, 1, 1);   // first shader writes to image
4
5
6
7
8
9
10
11
12
13
14     ...
15     vkCmdDispatch(commandBuffer, NUM_WORKGROUPS, 1, 1);   // second shader reads from same image
16 }
```

How do we ensure that the first shader finishes executing and that the data is available and visible before the second shader starts?

Pipeline Barriers

- Pipeline barriers synchronize previously issued workload with subsequently issued workload.

- Define three different types of barriers
  - **Global memory barriers**: affect all memory locations.
  - **Buffer memory barriers**: only affect the specified range of a buffer.
  - **Image memory barriers**: only affect the specified image subresource.

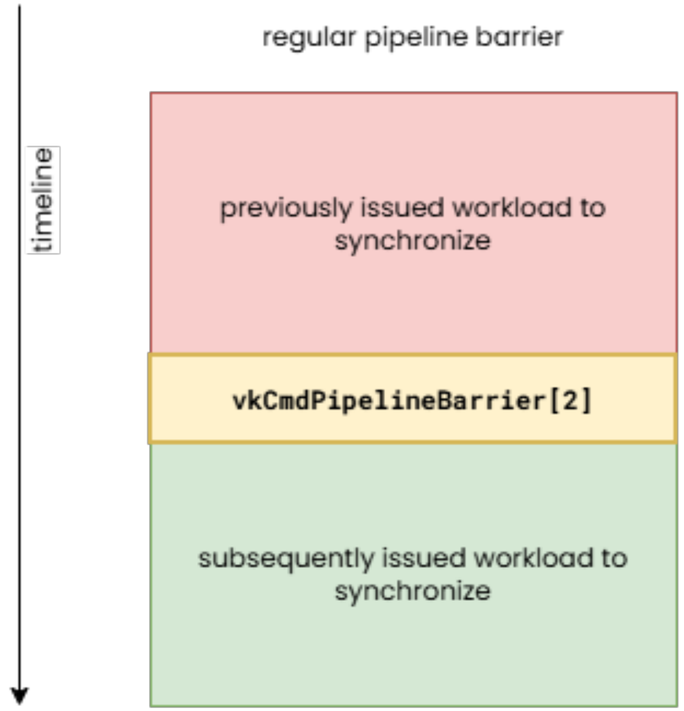


Image by RasterGrid

Pipeline Barriers

```
1  void vkCmdPipelineBarrier(
2      VkCommandBuffer                 commandBuffer,
3      VkPipelineStageFlags            srcStageMask,
4      VkPipelineStageFlags            dstStageMask,
5      VkDependencyFlags               dependencyFlags,
6      uint32_t                        memoryBarrierCount,
7      const VkMemoryBarrier*          pMemoryBarriers,
8      uint32_t                        bufferMemoryBarrierCount,
9      const VkBufferMemoryBarrier*    pBufferMemoryBarriers,
10     uint32_t                        imageMemoryBarrierCount,
11     const VkImageMemoryBarrier*     pImageMemoryBarriers);
```

- `srcStageMask`/`dstStageMask` : stages of commands before/after the barrier from which accesses should be included.
- Each `Vk * MemoryBarrier` adds a memory dependency between the source and destination stages.
- Without any memory barriers: execution dependency only.

Pipeline Stages

```
typedef enum VkPipelineStageFlagBits {
    VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT = 0x00000001,
    VK_PIPELINE_STAGE_DRAW_INDIRECT_BIT = 0x00000002,
    VK_PIPELINE_STAGE_VERTEX_INPUT_BIT = 0x00000004,
    VK_PIPELINE_STAGE_VERTEX_SHADER_BIT = 0x00000008,
    VK_PIPELINE_STAGE_TESSELLATION_CONTROL_SHADER_BIT = 0x0000
    VK_PIPELINE_STAGE_TESSELLATION_EVALUATION_SHADER_BIT = 0x00
    VK_PIPELINE_STAGE_GEOMETRY_SHADER_BIT = 0x00000040,
    VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT = 0x00000080,
    VK_PIPELINE_STAGE_EARLY_FRAGMENT_TESTS_BIT = 0x00000100,
    VK_PIPELINE_STAGE_LATE_FRAGMENT_TESTS_BIT = 0x00000200,
    VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT = 0x00000400,
    VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT = 0x00000800,
    VK_PIPELINE_STAGE_TRANSFER_BIT = 0x00001000,
    VK_PIPELINE_STAGE_BOTTOM_OF_PIPE_BIT = 0x00002000,
    VK_PIPELINE_STAGE_HOST_BIT = 0x00004000,
    VK_PIPELINE_STAGE_ALL_GRAPHICS_BIT = 0x00008000,
    VK_PIPELINE_STAGE_ALL_COMMANDS_BIT = 0x00010000,
    ...
  } VkPipelineStageFlagBits;
```

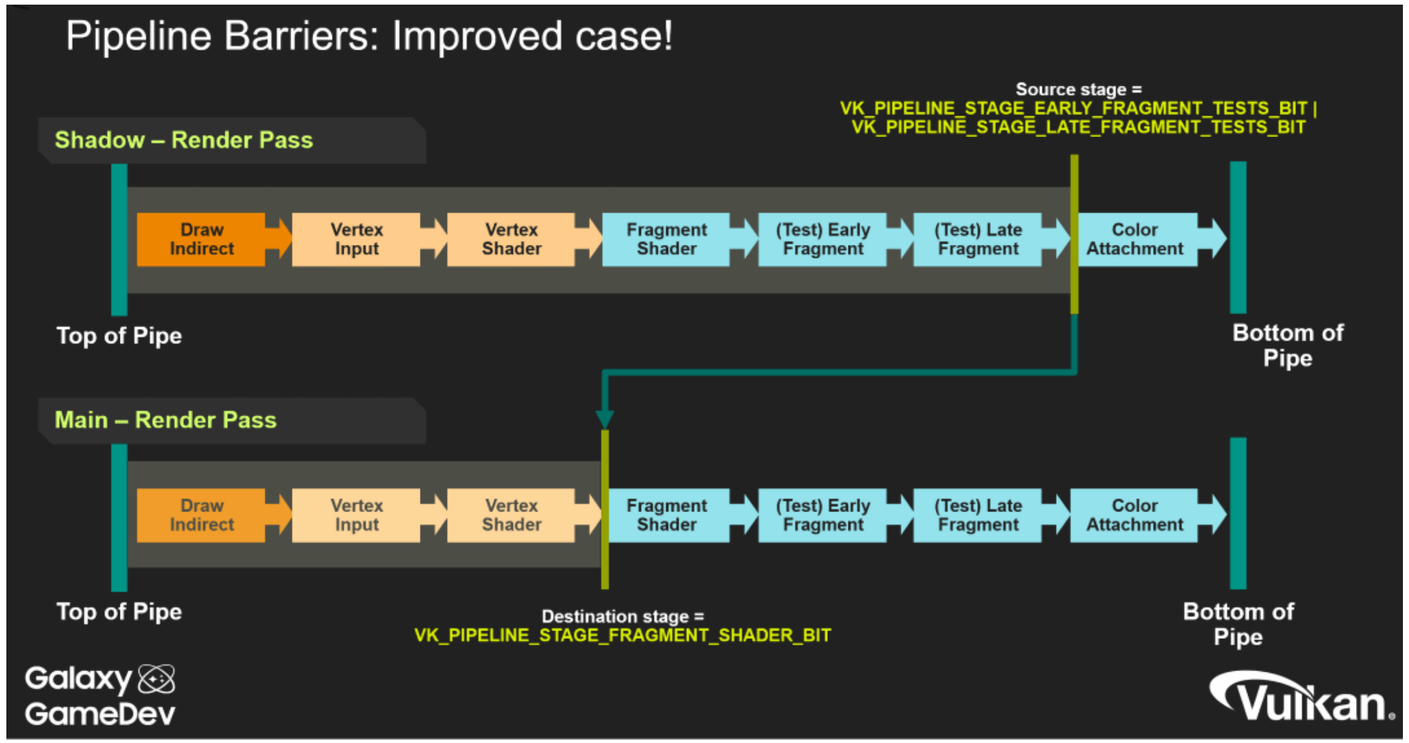
- Logical stages of pipelines.
- Barriers include memory accesses within specified stages.
- Compute pipeline:

  `TOP` → `DRAW_INDIRECT` → `COMPUTE_SHADER` → `BOTTOM`
- Graphics pipeline:

  `TOP` → ... → `VERTEX_SHADER` → ... → `FRAGMENT_SHADER` → ... → `COLOR_ATTACHMENT_OUTPUT` → `BOTTOM`

Pipeline Stages

- Explicit pipeline stages allow pipelines to run partially while still waiting for resources needed in later stages.
- Example: While the fragment stage is waiting for a texture written by a previous render pass, the vertex stage can begin executing.



https://docs.vulkan.org/guide/latest/synchronization.html

Global Memory Barriers

```
typedef struct VkMemoryBarrier {
    VkStructureType    sType;
    const void*        pNext;
    VkAccessFlags      srcAccessMask;
    VkAccessFlags      dstAccessMask;
} VkMemoryBarrier;
```

```
typedef enum VkAccessFlagBits {
    VK_ACCESS_INDIRECT_COMMAND_READ_BIT = 0x00000001,
    VK_ACCESS_INDEX_READ_BIT = 0x00000002,
    VK_ACCESS_VERTEX_ATTRIBUTE_READ_BIT = 0x00000004,
    VK_ACCESS_UNIFORM_READ_BIT = 0x00000008,
    VK_ACCESS_INPUT_ATTACHMENT_READ_BIT = 0x00000010,
    VK_ACCESS_SHADER_READ_BIT = 0x00000020,
    VK_ACCESS_SHADER_WRITE_BIT = 0x00000040,
    VK_ACCESS_COLOR_ATTACHMENT_READ_BIT = 0x00000080,
    VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT = 0x00000100,
    VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_READ_BIT = 0x00000200,
    VK_ACCESS_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT = 0x00000400,
    VK_ACCESS_TRANSFER_READ_BIT = 0x00000800,
    ...
} VkAccessFlagBits;
```

- Global barriers apply to all memory accesses in the specified stages.
- Access flags narrow down the types of access (e.g. only resources read/written).
- Most access types only apply to one pipeline stage.
- `TOP_OF_PIPE` and `BOTTOM_OF_PIPE` perform no memory accesses. Access mask must be zero.

Example: Two Compute Shaders

```
1  void recordCommandBuffer(VkCommandBuffer commandBuffer) {}
2      ...
3      vkCmdDispatch(commandBuffer, NUM_WORKGROUPS, 1, 1);   // first shader writes to image
4
5      const VkMemoryBarrier barrier{
6          .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER,
7          .srcAccessMask = VK_ACCESS_SHADER_WRITE_BIT,
8          .dstAccessMask = VK_ACCESS_SHADER_READ_BIT,
9      };
10     vkCmdPipelineBarrier(commandBuffer,
11         VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT,
12         VK_PIPELINE_STAGE_COMPUTE_SHADER_BIT,
13         0,                                                // dependencyFlags
14         1, &barrier,                                      // memory barriers
15         0, nullptr,                                       // buffer memory barriers
16         0, nullptr                                        // image memory barriers
17     );
18
19     ...
20     vkCmdDispatch(commandBuffer, NUM_WORKGROUPS, 1, 1);   // second shader reads from same image
21 }
```

Buffer Memory Barriers

```
typedef struct VkBufferMemoryBarrier {
    VkStructureType    sType;
    const void*        pNext;
    VkAccessFlags      srcAccessMask;
    VkAccessFlags      dstAccessMask;
    uint32_t           srcQueueFamilyIndex;
    uint32_t           dstQueueFamilyIndex;
    VkBuffer           buffer;
    VkDeviceSize       offset;
    VkDeviceSize       size;
} VkBufferMemoryBarrier;
```

- Restricts the memory dependency to a range of one buffer.
- In practice, current desktop drivers do not benefit from per-buffer barriers. Global memory barriers are simpler and equally fast.
- Buffer memory barriers are required when transferring ownership between queues. To not transfer, set queue families to `VK_QUEUE_FAMILY_IGNORED`.

Image Memory Barriers

```
typedef struct VkImageMemoryBarrier {
    VkStructureType            sType;
    const void*                pNext;
    VkAccessFlags              srcAccessMask;
    VkAccessFlags              dstAccessMask;
    VkImageLayout              oldLayout;
    VkImageLayout              newLayout;
    uint32_t                   srcQueueFamilyIndex;
    uint32_t                   dstQueueFamilyIndex;
    VkImage                    image;
    VkImageSubresourceRange    subresourceRange;
} VkImageMemoryBarrier;
```

- Restricts the memory dependency to a subresource range of one image.
- A **layout transition** is performed if `oldLayout != newLayout`.
- `oldLayout` must match the current layout of the image.
- `oldLayout` may be `UNDEFINED`, which discards the contents.
- Image barriers are required whenever the layout changes.

Layout Transitions

```
void transitionImageLayout(
    VkCommandBuffer commandBuffer, VkImage image, VkFormat format,
    VkImageLayout oldLayout, VkImageLayout newLayout
) {
    VkAccessFlags srcAccessMask = ...;
    VkAccessFlags dstAccessMask = ...;
    VkPipelineStageFlags srcStage = ...;
    VkPipelineStageFlags dstStage = ...;

    const VkImageMemoryBarrier barrier{
        .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER,
        .srcAccessMask = srcAccessMask,
        .dstAccessMask = dstAccessMask,
        .oldLayout = oldLayout,
        .newLayout = newLayout,
        .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
        .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
        .image = image,
        .subresourceRange = { VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1 },
    };
    vkCmdPipelineBarrier(commandBuffer, srcStage, dstStage, 0,
        0, nullptr, 0, nullptr, 1, &barrier);
}
```

- Small helper functions for layout transitions can be extremely useful.
- Stages and access flags can usually be inferred from the image layout. Example:
  `TRANSFER_DST_OPTIMAL`:
  →`ACCESS_TRANSFER_WRITE`
  →`PIPELINE_STAGE_TRANSFER`
- Manual layout transitions are usually only used during data upload and between compute shaders.

Example: Image Upload

```
1  // access NONE, stage TOP_OF_PIPE -> access TRANSFER_WRITE, stage TRANSFER
2  transitionImageLayout(
3      commandBuffer, image, imageFormat,
4      VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL);
5
6  const VkBufferImageCopy region{
7      .bufferOffset = 0,
8      .bufferRowLength = 0,
9      .bufferImageHeight = 0,
10     .imageSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
11     .imageOffset = {0, 0, 0},
12     .imageExtent = imageExtent,
13 };
14 vkCmdCopyBufferToImage(
15     commandBuffer, stagingBuffer, image, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1, &region);
16
17 // access TRANSFER_WRITE, stage TRANSFER -> access SHADER_READ, stage FRAGMENT_SHADER
18 transitionImageLayout(
19     commandBuffer, image, imageFormat,
20     VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL);
```

Example: Runtime Mipmap Generation

- Vulkan has no equivalent to `glGenerateMipmap`.
- Two options:
  - Store mipmaps on disk (preferred, especially when using compressed formats).
  - Generate from base image on load time.
- General procedure:
  - Load base level into buffer and copy to image
  - Transition current level to `TRANSFER_SRC` and next level to `TRANSFER_DST`
  - Blit higher mip level to next lower mip level
  - Repeat until all mip levels created
  - Transition all layers to `SHADER_READ_ONLY_OPTIMAL`

Blitting Images

```
void vkCmdBlitImage(
    VkCommandBuffer         commandBuffer,
    VkImage                 srcImage,
    VkImageLayout           srcImageLayout,
    VkImage                 dstImage,
    VkImageLayout           dstImageLayout,
    uint32_t                regionCount,
    const VkImageBlit*      pRegions,
    VkFilter                filter);
```

```
typedef struct VkImageBlit {
    VkImageSubresourceLayers    srcSubresource;
    VkOffset3D                  srcOffsets[2];
    VkImageSubresourceLayers    dstSubresource;
    VkOffset3D                  dstOffsets[2];
} VkImageBlit;
```

- Copies a region with scaling, filtering and format conversion.
- Source and destination can be same image, but different subresources.
- Requires support on the format and a queue with graphics capability.
- `srcOffsets[0]` and `srcOffsets[1]` are the corners of the source quad.
- `dstOffsets[0]` and `dstOffsets[1]` are the corners of the destination quad.

TOP_OF_PIPE and BOTTOM_OF_PIPE

- As a `srcStageMask`:
  - `TOP_OF_PIPE` waits for earlier commands to begin execution.
  - `BOTTOM_OF_PIPE` waits for earlier commands to complete execution.
- As a `dstStageMask`:
  - `TOP_OF_PIPE` blocks later commands from beginning execution entirely.
  - `BOTTOM_OF_PIPE` blocks nothing at all.
- Access mask must be zero:
  → No memory dependencies are created, only execution dependencies.
- Main use case:
  → `TOP_OF_PIPE` on a freshly created image from `VK_IMAGE_LAYOUT_UNDEFINED`

Dependency by Region

```
1     VkDependencyFlags                dependencyFlags,
```

```
1 typedef enum VkDependencyFlagBits {
2     VK_DEPENDENCY_BY_REGION_BIT = 0x00000001,
3     ...
4 } VkDependencyFlagBits;
```

- Reduces the scope of the dependencies.
- Example: A later fragment shader reads the results from an earlier fragment shader, but only at the exact same fragment position.
- Allows for additional optimization, especially on tile-based GPUs.

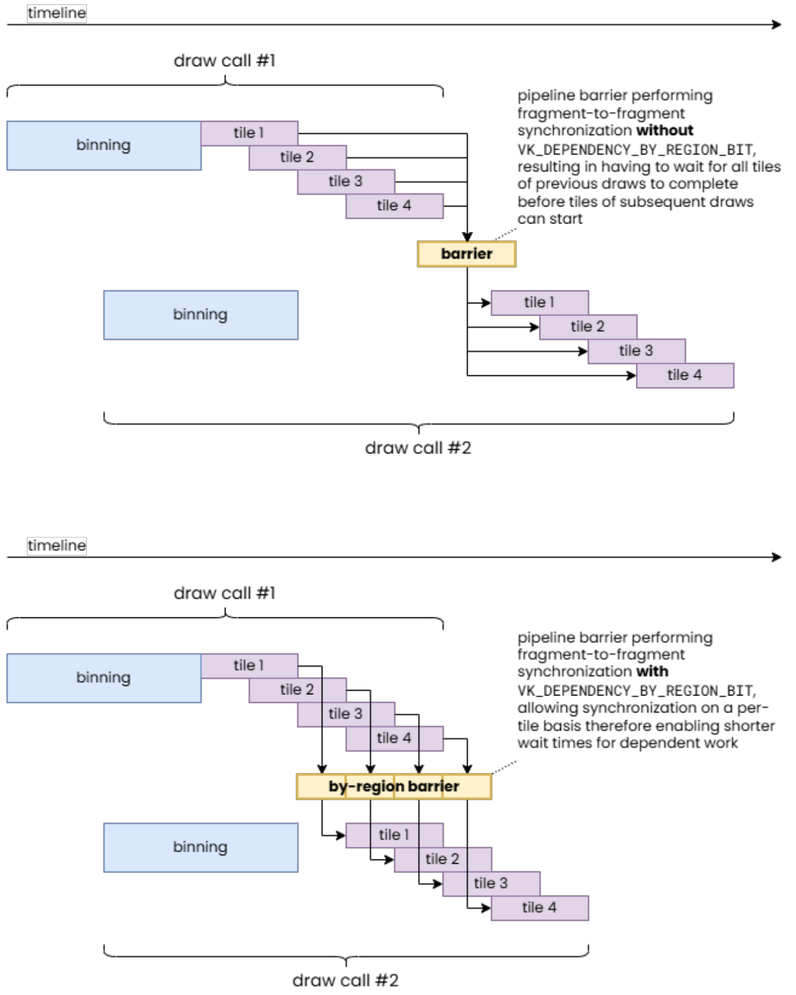


Image by RasterGrid

Events

- Similar to pipeline barriers, but split into two.
- `vkCmdSetEvent` sets dependencies of previous commands.
- `vkCmdWaitEvents` then has later commands wait for those dependencies.
- Commands between the two calls are not affected.
- Rarely needed. Pipeline barriers are usually enough.

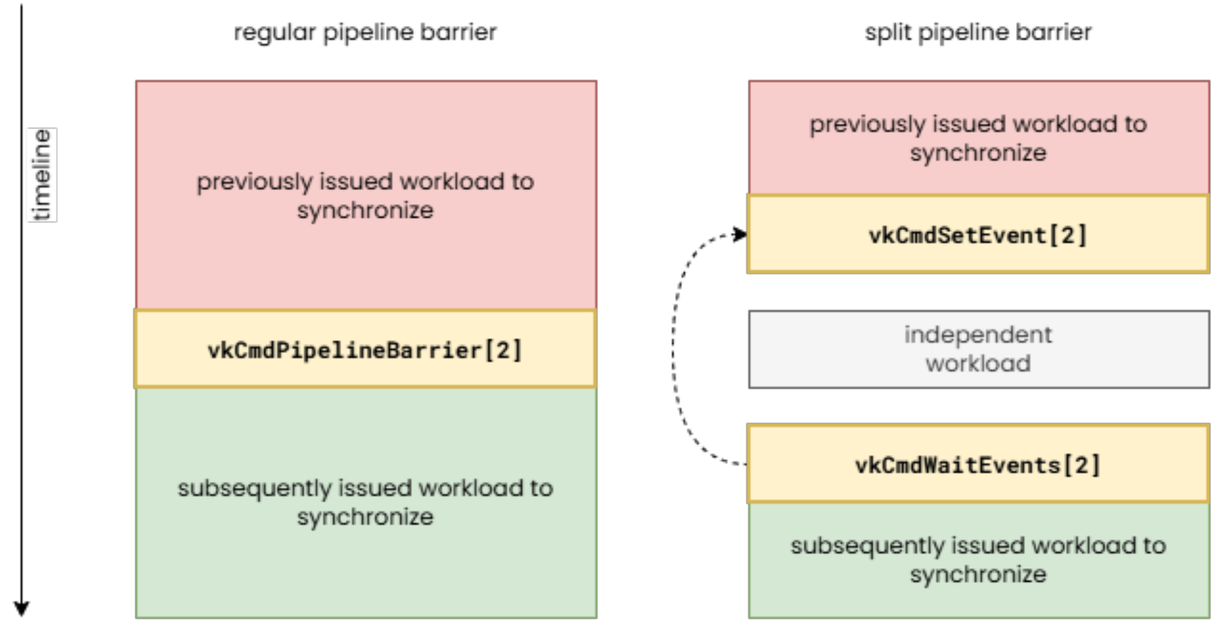


Image by RasterGrid

Events

```
VkResult vkCreateEvent(
    VkDevice                        device,
    const VkEventCreateInfo*        pCreateInfo,
    const VkAllocationCallbacks*    pAllocator,
    VkEvent*                        pEvent);
```

```
void vkCmdSetEvent(
    VkCommandBuffer                 commandBuffer,
    VkEvent                         event,
    VkPipelineStageFlags            stageMask);
```

```
void vkCmdWaitEvents(
    VkCommandBuffer                 commandBuffer,
    uint32_t                        eventCount,
    const VkEvent*                  pEvents,
    VkPipelineStageFlags            srcStageMask,
    VkPipelineStageFlags            dstStageMask,
    uint32_t                        memoryBarrierCount,
    const VkMemoryBarrier*          pMemoryBarriers,
    uint32_t                        bufferMemoryBarrierCount,
    const VkBufferMemoryBarrier*    pBufferMemoryBarriers,
    uint32_t                        imageMemoryBarrierCount,
    const VkImageMemoryBarrier*     pImageMemoryBarriers);
```

Semaphores

```
1 const VkSemaphoreCreateInfo semaphoreInfo{
2     .sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO,
3 };
4 VK_CHECK(vkCreateSemaphore(device, &semaphoreInfo, nullptr, &semaphore));
```

- Two states: signaled and unsignaled.
- Semaphores are signaled when all commands before and all memory dependencies have been resolved.
- Every signal must be followed by exactly one wait.
- The CPU cannot wait on or query semaphores in Vulkan 1.0.
- Used for dependencies between submissions on different queues and with the presentation engine.

Von Schermuuser, CC BY-SA 3.0

Fences

```
1 const VkFenceCreateInfo fenceInfo{
2     .sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO,
3 };
4 VK_CHECK(vkCreateFence(device, &fenceInfo, nullptr, &fence));
```

- Similar to semaphores. Signaled by the GPU,
  but queried by the CPU.
- Can be polled (`vkGetFenceStatus`) or waited on (`vkWaitForFences`).
- Must be reset manually (`vkResetFences`) before used again.
- After the signal, all previous commands are no longer pending
  and all resources used by them can be reused or destroyed.

Voice of Clam - public domain

Queue Submission

```
1  VkResult vkQueueSubmit(
2      VkQueue                     queue,
3      uint32_t                    submitCount,
4      const VkSubmitInfo*         pSubmits,
5      VkFence                     fence);
```

```
1  typedef struct VkSubmitInfo {
2      VkStructureType             sType;
3      const void*                 pNext;
4      uint32_t                    waitSemaphoreCount;
5      const VkSemaphore*          pWaitSemaphores;
6      const VkPipelineStageFlags* pWaitDstStageMask;
7      uint32_t                    commandBufferCount;
8      const VkCommandBuffer*      pCommandBuffers;
9      uint32_t                    signalSemaphoreCount;
10     const VkSemaphore*          pSignalSemaphores;
11 } VkSubmitInfo;
```

- The GPU waits for the semaphores in `pWaitSemaphores`, before it executes any stages specified in the respective `pWaitDstStageMask`.
- When all command buffers have executed, `pSignalSemaphores` are signaled.
- Finally, `fence` is signaled.

Waiting Idle

```
1 VkResult vkQueueWaitIdle(VkQueue queue);
2 VkResult vkDeviceWaitIdle(VkDevice device);
```

- Blocks the CPU until all work on a queue or on the whole device is complete.
- Equivalent to submitting with a fence and waiting on it.
- Acceptable use cases: during loading and for debugging.
- Required before destroying resources at shutdown.
- Never wait idle on every frame.

Recap - Synchronization

- Synchronization in Vulkan is mostly manual.
- Execution dependencies only protect against write-after-read.
- Memory dependencies also make written data available and visible.
- Pipeline barriers synchronize at specific points within pipelines.
- Image memory barriers perform layout transitions.
- Events are barriers that allow independent workload in between synchronization.
- Semaphores synchronize between different queues and with the presentation engine.
- Fences synchronize between the GPU and the CPU.

Swapchain

Window System Integration

- Core Vulkan does not define any windows or displays.
- Presentation support is provided by **Window System Integration (WSI)** extensions:
  - `VK_KHR_surface`: instance extension for platform-independent surface object
  - `VK_KHR_win32_surface`, `VK_KHR_xlib_surface`, `VK_KHR_wayland_surface`, ...: instance extensions for platform-specific surface creation
  - `VK_KHR_swapchain`: device extension for the swapchain itself
- These extensions are supported by all desktop drivers.

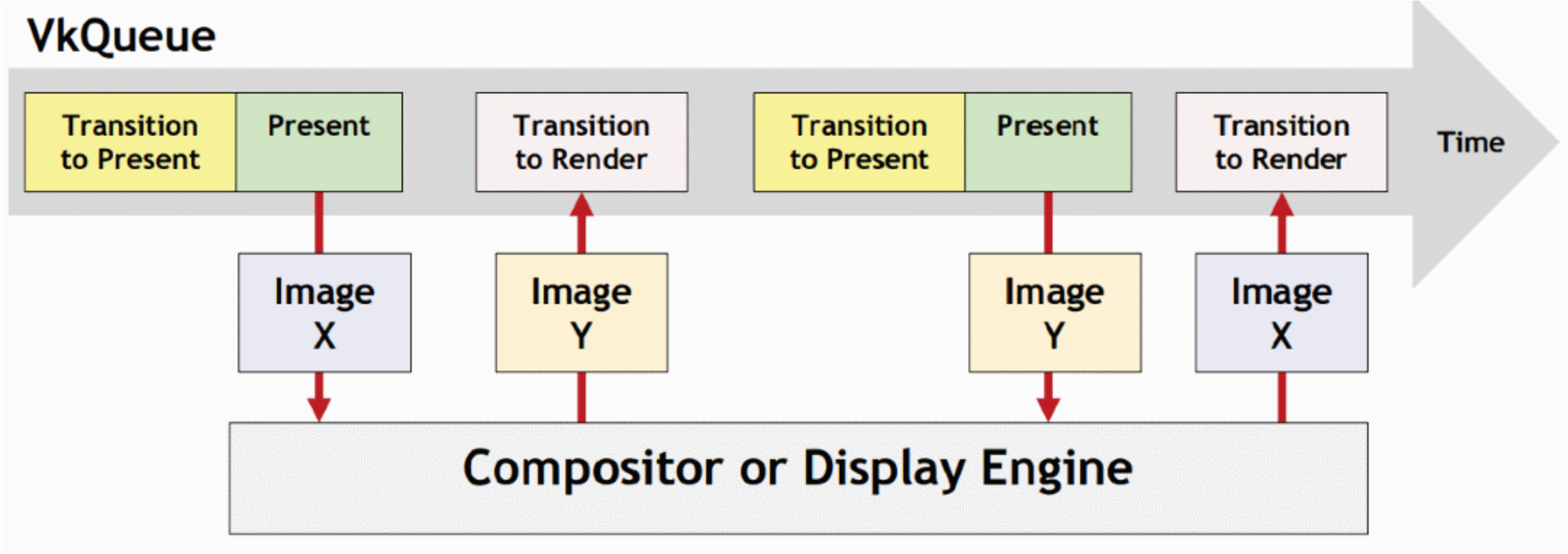
- **Surface**: an abstraction of the operating system window to present to.
- **Swapchain**: a set of images to render to, owned by the presentation engine.

Enabling WSI with GLFW

```
1  glfwInit();
2  glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);   // no OpenGL context
3  GLFWwindow* window = glfwCreateWindow(1280, 720, "Vulkan", nullptr, nullptr);
4
5  // contains VK_KHR_surface and the platform surface extension
6  uint32_t extCount = 0;
7  const char** extensions = glfwGetRequiredInstanceExtensions(&extCount);
8
9  // create instance with 'extensions' enabled
10 ...
11
12 VkSurfaceKHR surface;
13 VK_CHECK(glfwCreateWindowSurface(instance, window, nullptr, &surface));
```

Swapchain

- A series of images that allow presenting of rendering results to a surface.



https://docs.vulkan.org/guide/latest/wsi.html

Swapchain Creation

```
typedef struct VkSwapchainCreateInfoKHR {
    VkStructureType                  sType;
    const void*                      pNext;
    VkSwapchainCreateFlagsKHR        flags;
    VkSurfaceKHR                     surface;
    uint32_t                         minImageCount;
    VkFormat                         imageFormat;
    VkColorSpaceKHR                  imageColorSpace;
    VkExtent2D                       imageExtent;
    uint32_t                         imageArrayLayers;
    VkImageUsageFlags                imageUsage;
    VkSharingMode                    imageSharingMode;
    uint32_t                         queueFamilyIndexCount;
    const uint32_t*                  pQueueFamilyIndices;
    VkSurfaceTransformFlagBitsKHR    preTransform;
    VkCompositeAlphaFlagBitsKHR      compositeAlpha;
    VkPresentModeKHR                 presentMode;
    VkBool32                         clipped;
    VkSwapchainKHR                   oldSwapchain;
} VkSwapchainCreateInfoKHR;
```

- Creates at least `minImageCount` images with the specified parameters from the surface.
- Swapchains usually only support a subset of image parameters available for the device.
- When recreating (window resize), the `oldSwapchain` is passed.

Surface Capabilities

```
VkResult vkGetPhysicalDeviceSurfaceCapabilitiesKHR(
    VkPhysicalDevice                physicalDevice,
    VkSurfaceKHR                    surface,
    VkSurfaceCapabilitiesKHR*       pSurfaceCapabilities);
```

```
typedef struct VkSurfaceCapabilitiesKHR {
    uint32_t                        minImageCount;
    uint32_t                        maxImageCount;
    VkExtent2D                      currentExtent;
    VkExtent2D                      minImageExtent;
    VkExtent2D                      maxImageExtent;
    uint32_t                        maxImageArrayLayers;
    VkSurfaceTransformFlagsKHR      supportedTransforms;
    VkSurfaceTransformFlagBitsKHR   currentTransform;
    VkCompositeAlphaFlagsKHR        supportedCompositeAlpha;
    VkImageUsageFlags               supportedUsageFlags;
} VkSurfaceCapabilitiesKHR;
```

- Supports between `minImageCount` and `maxImageCount` images. `maxImageCount` of zero means no upper limit.
- `currentExtent`: current size of the surface. `0xFFFFFFFF` means the application chooses the size.
- Array layers are mainly used for stereoscopic applications.
- Transforms control rotation of display. Mainly relevant on mobile.
- `supportedUsageFlags` always contains `VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT`.

Surface Formats

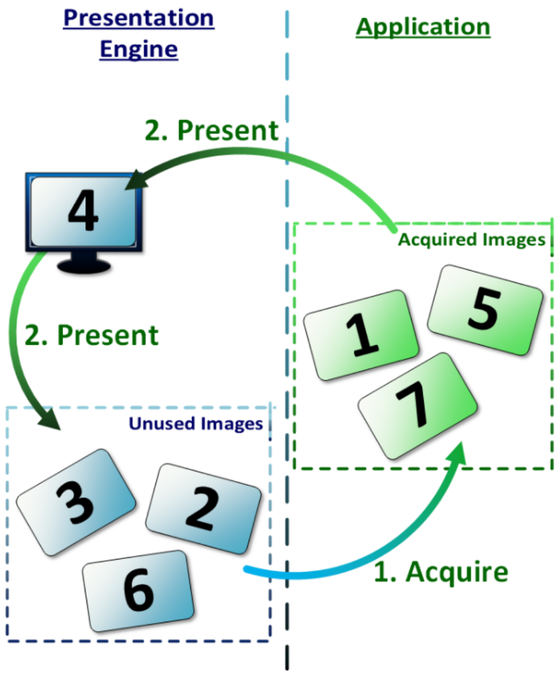
```
1 VkResult vkGetPhysicalDeviceSurfaceFormatsKHR(
2     VkPhysicalDevice       physicalDevice,
3     VkSurfaceKHR           surface,
4     uint32_t*              pSurfaceFormatCount,
5     VkSurfaceFormatKHR*    pSurfaceFormats);
```

```
1 typedef struct VkSurfaceFormatKHR {
2     VkFormat               format;
3     VkColorSpaceKHR        colorSpace;
4 } VkSurfaceFormatKHR;
```

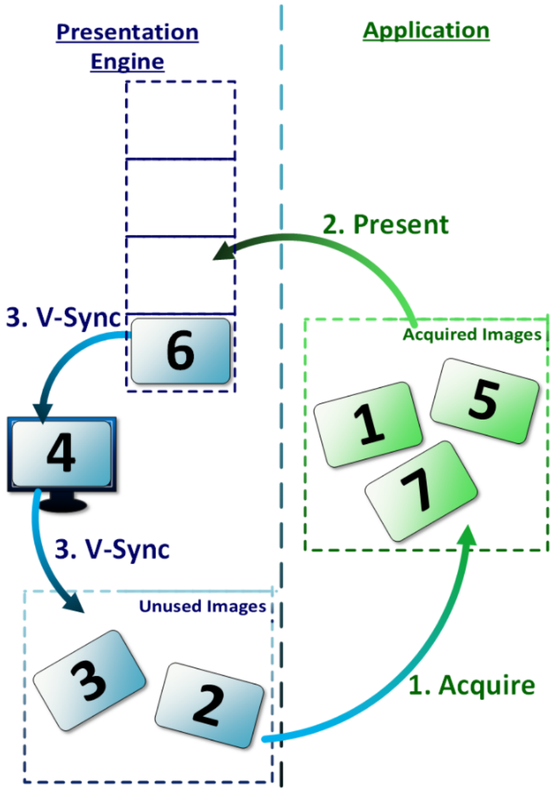
- Typically `B8G8R8A8_UNORM` and `B8G8R8A8_SRGB` with `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR` are supported SDR options.
- The color space describes how the presentation engine interprets the values. Usually in SRGB.
- The format defines how the values are written to the images.

Present Modes

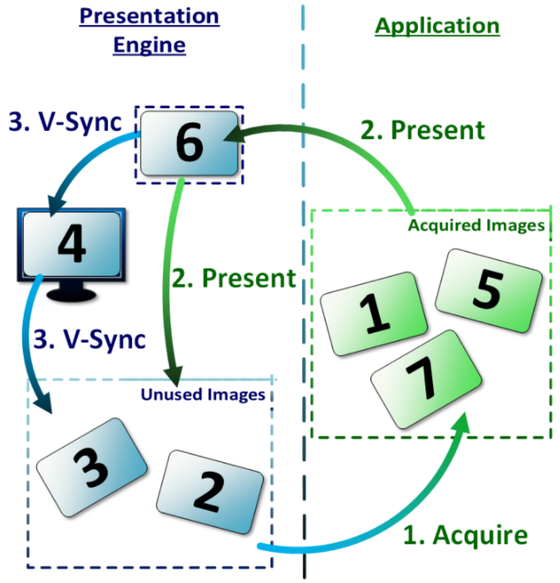
Immediate



FIFO



Mailbox



API without Secrets: Introduction to Vulkan* Part 2 - Intel

Present Modes

```
1 typedef enum VkPresentModeKHR {
2     VK_PRESENT_MODE_IMMEDIATE_KHR = 0,
3     VK_PRESENT_MODE_MAILBOX_KHR = 1,
4     VK_PRESENT_MODE_FIFO_KHR = 2,
5     VK_PRESENT_MODE_FIFO_RELAXED_KHR = 3,
6     ...
7 } VkPresentModeKHR;
```

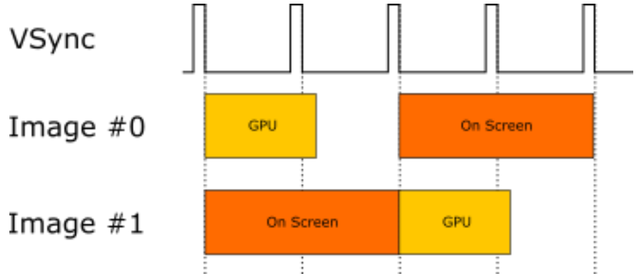
```
1 VkResult vkGetPhysicalDeviceSurfacePresentModesKHR(
2     VkPhysicalDevice    physicalDevice,
3     VkSurfaceKHR        surface,
4     uint32_t*           pPresentModeCount,
5     VkPresentModeKHR*   pPresentModes);
```

- `FIFO`: queue of images, first is taken at each vertical blank. V-sync, no tearing. Will repeat images if application is late.
- `FIFO_RELAXED`: like FIFO, but presents immediately if the application was late.
- `MAILBOX`: only one image waits for the vertical blank, older ones are replaced. No tearing, low latency, renders more frames than displayed.
- `IMMEDIATE`: images are displayed immediately, lowest latency, tearing.
- Only `FIFO` is guaranteed to be supported.

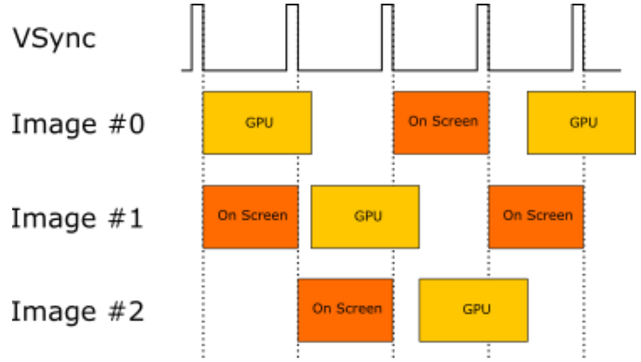
Number of Swapchain Images

- Two images is usual minimum for swapchain.
- With double buffering, drawing only starts when previous display frame finishes.
- Double buffering fails when frame takes longer than VSync period.
  → Immediately forces to next integer multiple of VSync.
- Triple buffering is a sensible default.

Double Buffering



Triple Buffering



https://docs.vulkan.org/samples/latest/samples/performance/hpp_swapchain_images/README.html

Swapchain Creation

```
const VkSwapchainCreateInfoKHR swapchainCreateInfo{          // ↓ chosen from
    .sType = VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR,
    .surface = surface,
    .minImageCount = chosenImageCount,                       // vkGetPhysicalDeviceSurfaceCapabilitiesKHR
    .imageFormat = chosenSurfaceFormat.format,               // vkGetPhysicalDeviceSurfaceFormatsKHR
    .imageColorSpace = chosenSurfaceFormat.colorSpace,       // vkGetPhysicalDeviceSurfaceFormatsKHR
    .imageExtent = chosenExtent,                             // vkGetPhysicalDeviceSurfaceCapabilitiesKHR
    .imageArrayLayers = 1,
    .imageUsage = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT,
    .imageSharingMode = VK_SHARING_MODE_EXCLUSIVE,
    .preTransform = currentTransform,                        // vkGetPhysicalDeviceSurfaceCapabilitiesKHR
    .compositeAlpha = VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR,
    .presentMode = chosenPresentMode,                        // vkGetPhysicalDeviceSurfacePresentModesKHR
    .clipped = VK_TRUE,
};
VK_CHECK(vkCreateSwapchainKHR(device, &swapchainCreateInfo, nullptr, &swapchain));
```

Swapchain Images

```
1 uint32_t count = 0;
2 vkGetSwapchainImagesKHR(device, swapchain, &count, nullptr);
3 std::vector<VkImage> images(count);
4 vkGetSwapchainImagesKHR(device, swapchain, &count, images.data());
```

- The images are created and owned by the swapchain.
  No memory can be bound to them and they can not be destroyed.
- Start in `UNDEFINED` layout.
- Image views can and usually have to be created for them.

Acquire and Present

```
1  VkResult vkAcquireNextImageKHR(
2      VkDevice           device,
3      VkSwapchainKHR     swapchain,
4      uint64_t           timeout,
5      VkSemaphore        semaphore,
6      VkFence            fence,
7      uint32_t*          pImageIndex);
```

```
1  VkResult vkQueuePresentKHR(
2      VkQueue                     queue,
3      const VkPresentInfoKHR*     pPresentInfo);
4
5  typedef struct VkPresentInfoKHR {
6      VkStructureType             sType;
7      const void*                 pNext;
8      uint32_t                    waitSemaphoreCount;
9      const VkSemaphore*          pWaitSemaphores;
10     uint32_t                    swapchainCount;
11     const VkSwapchainKHR*       pSwapchains;
12     const uint32_t*             pImageIndices;
13     VkResult*                   pResults;
14 } VkPresentInfoKHR;
```

- Acquire returns index of next image to use. Images are not necessarily returned in order.
- The function may return while the engine is still reading the image. The semaphore and/or fence are signaled when it can actually be written.
- `vkQueuePresentKHR` queues the image for presenting after waiting for the given semaphores.
- The image must be in the layout `PRESENT_SRC_KHR` when presented.

Swapchain Recreation

- The swapchain has to be recreated when the surface changes, e.g. on window resize.
- `vkAcquireNextImageKHR`/`vkQueuePresentKHR` return:
  - `VK_ERROR_OUT_OF_DATE_KHR`: the swapchain can no longer be used for presentation.
  - `VK_SUBOPTIMAL_KHR`: the swapchain no longer matches the surface, but can still be used.
- Window resize callback of windowing library is usually more reliable.
- When the swapchain is recreated, all dependent resources (image views, semaphores), and size-dependent resources (render targets) have to be rebuilt.
- Minimized windows have an extent of 0×0. No swapchain can be created from this. Application should instead wait for the next resize.

Recap - Swapchain

- Presentation is provided by the WSI extensions `VK_KHR_surface` and `VK_KHR_swapchain`.
- Surface creation is platform dependent. Windowing libraries abstract this away.
- The surface’s capabilities have to be queried before creating the swapchain.
- Presentation engines typically display content in SRGB.
- Swapchain images are owned by the swapchain and start in `UNDEFINED` layout.
- Acquiring and presenting images is synchronized by semaphores.
- The swapchain has to be recreated when the window changes.

Next Time

- The graphics pipeline: vertex input, rasterization, render passes and framebuffers